

# Desarrollo de un sistema embebido para la gestión y monitoreo de cultivos verticales

Ing. José Luis Krüger

**Carrera de Especialización en Sistemas Embebidos**

**Director:** Esp. Ing. Luis Mariano Campos (FIUBA)

**Jurados:**

Esp. Ing. Roberto Oscar Axt (FIUBA)  
Esp. Ing. Agustín Jesús Vazquez (FIUBA)  
Ing. Franco Chiesa Docampo (FIUBA)

*Ciudad de Buenos Aires, diciembre de 2025*



## *Resumen*

Este trabajo presenta el desarrollo, construcción, instalación y prueba de un dispositivo semiautomático de monitoreo y gestión para un cultivo hidropónico vertical. El sistema mejora la eficiencia del cultivo al minimizar los desperdicios y aumentar la productividad en comparación con los métodos tradicionales. Este avance representa un paso clave hacia la sostenibilidad y la optimización de la producción de alimentos en el futuro. Su implementación requirió la aplicación de conocimientos en electrónica, programación, sistemas operativos de tiempo real, testing de software, diseño y ensamblaje de PCB, integración de sensores y protocolos de comunicación.



## *Agradecimientos*

Agradezco profundamente a mi familia por su apoyo durante todo el desarrollo de este trabajo y de mi formación profesional. Su estímulo y paciencia han sido fundamentales para alcanzar esta meta.

A los miembros del jurado, agradezco su tiempo y dedicación para evaluar este trabajo. Sus conocimientos y experiencia son invaluable.

Un especial reconocimiento a mi director de tesis, cuya guía, predisposición y consejo han sido fundamentales para la culminación exitosa de esta empresa.

A los profesores del CESE, cuyo conocimiento y orientación han sido imprescindibles para la realización de este trabajo.

Finalmente, a todas las personas que de una u otra forma contribuyeron a la realización de este trabajo, mi más sincero agradecimiento por su apoyo y colaboración.



# Índice general

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Resumen</b>                                        | <b>I</b> |
| <b>1. Introducción general</b>                        | <b>1</b> |
| 1.1. Agricultura vertical como solución sostenible    | 1        |
| 1.1.1. Tipos de cultivos verticales                   | 2        |
| Hidroponía                                            | 2        |
| Aeroponía                                             | 2        |
| Acuaponía                                             | 2        |
| 1.1.2. Sistemas de agricultura vertical más conocidos | 2        |
| Sistemas de torre                                     | 2        |
| Sistemas en rack o estanterías                        | 2        |
| Sistemas montados en pared                            | 3        |
| Sistemas de bastidor en A                             | 3        |
| Sistemas de contenedores                              | 3        |
| 1.2. Motivación                                       | 3        |
| 1.3. Estado del arte                                  | 4        |
| NIDO PRO                                              | 4        |
| Xiaomi Mi Flower Care Plant Sensor                    | 4        |
| Kit de Riego Hydro de Konyks                          | 5        |
| Smart 9 Pro de Click & Grow                           | 6        |
| 1.4. Objetivos y alcances                             | 6        |
| 1.4.1. Objetivos                                      | 6        |
| 1.4.2. Alcances                                       | 7        |
| <b>2. Introducción específica</b>                     | <b>9</b> |
| 2.1. Componentes principales del hardware             | 9        |
| 2.1.1. ESP32-DevKitC                                  | 9        |
| 2.1.2. Módulo sensor PH-4502C                         | 10       |
| 2.1.3. Módulo medidor de sólidos disueltos totales    | 10       |
| 2.1.4. Sensor de humedad capacitivo V2.0              | 11       |
| 2.1.5. Sensor de temperatura digital DS18B20          | 11       |
| 2.1.6. Sensor de luz ambiental digital BH1750         | 12       |
| 2.1.7. Raspberry Pi                                   | 12       |
| 2.2. Componentes principales del software             | 12       |
| 2.2.1. ESP-IDF                                        | 13       |
| 2.2.2. FreeRTOS                                       | 13       |
| 2.2.3. Ceedling                                       | 13       |
| 2.2.4. React                                          | 13       |
| 2.2.5. Docker                                         | 13       |
| 2.3. Protocolos de comunicación empleados             | 13       |
| 2.3.1. HTTP                                           | 14       |
| 2.3.2. I2C                                            | 14       |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 2.3.3.    | 1-Wire . . . . .                                       | 14        |
| <b>3.</b> | <b>Diseño e implementación</b>                         | <b>15</b> |
| 3.1.      | Diagrama de bloques . . . . .                          | 15        |
| 3.2.      | Arquitectura del firmware . . . . .                    | 16        |
| 3.2.1.    | Patrones . . . . .                                     | 16        |
|           | Arquitectura en capas . . . . .                        | 16        |
|           | Capa de abstracción de hardware . . . . .              | 17        |
|           | Control ambiental . . . . .                            | 17        |
| 3.2.2.    | Componentes . . . . .                                  | 17        |
| 3.3.      | Desarrollo del software . . . . .                      | 18        |
| 3.3.1.    | Gestión de la comunicación entre tareas . . . . .      | 18        |
| 3.3.2.    | Gestión de prioridades . . . . .                       | 19        |
| 3.3.3.    | Controladores de dispositivos . . . . .                | 19        |
| 3.3.4.    | Gestión de fallos . . . . .                            | 19        |
| 3.3.5.    | Servidor embebido y API REST . . . . .                 | 19        |
| 3.3.6.    | Lógica de negocio . . . . .                            | 20        |
| 3.3.7.    | Estructura del código . . . . .                        | 20        |
| 3.3.8.    | Gestion de la configuración . . . . .                  | 23        |
| 3.4.      | Desarrollo del hardware . . . . .                      | 23        |
| 3.4.1.    | Arquitectura general . . . . .                         | 23        |
| 3.4.2.    | Diseño electrónico . . . . .                           | 24        |
| 3.4.3.    | Alimentación . . . . .                                 | 26        |
| 3.4.4.    | Diseño del PCB y ensamblado . . . . .                  | 26        |
| 3.4.5.    | Consideraciones de seguridad y mantenimiento . . . . . | 27        |
| 3.4.6.    | Ensamblado final . . . . .                             | 27        |
| 3.5.      | Selección y calibración de sensores . . . . .          | 28        |
| 3.5.1.    | Selección de sensores . . . . .                        | 28        |
| 3.5.2.    | Calibración de sensores . . . . .                      | 29        |
| 3.6.      | Interfaz gráfica . . . . .                             | 30        |
| 3.6.1.    | Diseño de la interfaz de usuario . . . . .             | 30        |
| 3.6.2.    | Seguridad y autenticación . . . . .                    | 31        |
| 3.6.3.    | Monitoreo de sensores . . . . .                        | 32        |
| 3.6.4.    | Control de actuadores . . . . .                        | 32        |
| 3.6.5.    | Configuración del sistema . . . . .                    | 33        |
| 3.6.6.    | Logs del sistema . . . . .                             | 34        |
| 3.6.7.    | Compatibilidad y optimización . . . . .                | 34        |
| <b>4.</b> | <b>Ensayos y resultados</b>                            | <b>35</b> |
| 4.1.      | Pruebas del firmware . . . . .                         | 35        |
| 4.1.1.    | Pruebas unitarias . . . . .                            | 35        |
| 4.1.2.    | Pruebas de comunicación entre módulos . . . . .        | 36        |
|           | Arquitectura pub-sub . . . . .                         | 36        |
|           | API REST con Postman . . . . .                         | 36        |
|           | Gestión de logs . . . . .                              | 38        |
|           | Comunicación con la nube . . . . .                     | 38        |
|           | Estado del sistema y actuadores . . . . .              | 39        |
|           | Pruebas de condiciones de carrera . . . . .            | 39        |
|           | Manejo de configuración . . . . .                      | 39        |
|           | Conectividad de red . . . . .                          | 40        |
| 4.2.      | Pruebas del hardware . . . . .                         | 40        |



|                                                         |           |
|---------------------------------------------------------|-----------|
| 4.2.1. Verificación eléctrica . . . . .                 | 40        |
| 4.2.2. Validación funcional . . . . .                   | 41        |
| 4.2.3. Pruebas de protección . . . . .                  | 41        |
| 4.3. Pruebas de la interfaz gráfica . . . . .           | 41        |
| 4.3.1. Pruebas de autenticación . . . . .               | 41        |
| 4.3.2. Monitoreo y actualización de datos . . . . .     | 41        |
| 4.3.3. Control de actuadores . . . . .                  | 42        |
| 4.3.4. Configuración del sistema . . . . .              | 43        |
| 4.3.5. Registro de eventos . . . . .                    | 43        |
| 4.4. Pruebas de integración . . . . .                   | 44        |
| 4.5. Validación de requerimientos funcionales . . . . . | 45        |
| 4.6. Banco de pruebas y resultados obtenidos . . . . .  | 47        |
| 4.6.1. Resumen general . . . . .                        | 47        |
| <b>5. Conclusiones</b>                                  | <b>49</b> |
| 5.1. Conclusiones generales . . . . .                   | 49        |
| 5.2. Próximos pasos . . . . .                           | 49        |
| <b>Bibliografía</b>                                     | <b>51</b> |



# Índice de figuras

|                                                                                                                                                                                                 |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1. Ilustración del módulo NIDO PRO conectado a un sistema hidropónico horizontal <sup>1</sup> . . . . .                                                                                       | 4  |
| 1.2. Ilustración del dispositivo Xiaomi Mi Flower Care Plant Sensor y sus prestaciones <sup>2</sup> . . . . .                                                                                   | 5  |
| 1.3. Sistema de riego conectado Konyks <sup>3</sup> . . . . .                                                                                                                                   | 5  |
| 1.4. Módulo Smart Garden 9 PRO <sup>4</sup> . . . . .                                                                                                                                           | 6  |
| 2.1. ESP32-DevKitC V4 con el módulo ESP32-WROOM-32 integrado <sup>5</sup> . . . . .                                                                                                             | 9  |
| 2.2. Distribución de pines del módulo PH-4502C <sup>6</sup> . . . . .                                                                                                                           | 10 |
| 2.3. Ilustración del módulo sensor TDS meter v1.0 <sup>7</sup> . . . . .                                                                                                                        | 10 |
| 2.4. Módulo sensor de humedad capacitivo <sup>8</sup> . . . . .                                                                                                                                 | 11 |
| 2.5. Pinout del sensor DS18B20 y presentación del chip con su vaina protectora característica <sup>9</sup> . . . . .                                                                            | 12 |
| 2.6. Pinout del módulo de adaptación del sensor BH1750 <sup>10</sup> . . . . .                                                                                                                  | 12 |
| 3.1. Diagrama de bloques del sistema. . . . .                                                                                                                                                   | 15 |
| 3.2. Diagrama de módulos funcionales y sus interacciones. . . . .                                                                                                                               | 18 |
| 3.3. Diagrama simplificado del ciclo de ejecución del programa. . . . .                                                                                                                         | 21 |
| 3.4. Propagación de la configuración. . . . .                                                                                                                                                   | 23 |
| 3.5. Vista parcial del circuito de la placa centralizadora que muestra la interconexión de los módulos de TDS y pH con el microcontrolador y el circuito de acondicionamiento de señal. . . . . | 25 |
| 3.6. Circuito de control de la bomba con aislamiento galvánico y relé de potencia. . . . .                                                                                                      | 25 |
| 3.7. Vistas del circuito impreso (PCB) del sistema. . . . .                                                                                                                                     | 26 |
| 3.8. Montaje final del sistema. . . . .                                                                                                                                                         | 28 |
| 3.9. Diagrama de la estructura jerárquica de la aplicación. . . . .                                                                                                                             | 30 |
| 3.10. Diagrama de secuencia de renderizado de una vista. . . . .                                                                                                                                | 31 |
| 3.11. Diagrama de secuencia del proceso de autenticación. . . . .                                                                                                                               | 31 |
| 3.12. Diagrama de secuencia de la vista de monitoreo de sensores. . . . .                                                                                                                       | 32 |
| 3.13. Diagrama de secuencia de la vista de control de actuadores. . . . .                                                                                                                       | 33 |
| 3.14. Diagrama de secuencia de la vista de configuración. . . . .                                                                                                                               | 33 |
| 4.1. Reporte de pruebas unitarias de Ceedling. . . . .                                                                                                                                          | 35 |
| 4.2. Proceso de suscripción a canales. . . . .                                                                                                                                                  | 36 |
| 4.3. Manejo de desbordamiento del buffer. . . . .                                                                                                                                               | 36 |
| 4.4. Respuesta del servidor al intentar acceder sin autenticación. . . . .                                                                                                                      | 37 |
| 4.5. Pruebas de API con Postman - actualización de la configuración. . . . .                                                                                                                    | 39 |
| 4.6. Salida por consola durante la actualización de la configuración. . . . .                                                                                                                   | 40 |
| 4.7. Pantalla de inicio de sesión del sistema. . . . .                                                                                                                                          | 42 |
| 4.8. Vistas de monitoreo de sensores. . . . .                                                                                                                                                   | 42 |
| 4.9. Interfaz de control de actuadores. . . . .                                                                                                                                                 | 43 |
| 4.10. Pantallas de configuración. . . . .                                                                                                                                                       | 43 |

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| 4.11. Registro de eventos del sistema. . . . .                                                | 44 |
| 4.12. Vista de sensores con mediciones en tiempo real. . . . .                                | 45 |
| 4.13. Configuración del banco de pruebas con los componentes principales del sistema. . . . . | 47 |





# Capítulo 1

## Introducción general

En este capítulo se expone la necesidad que dio origen al desarrollo del trabajo. Se presenta el concepto de cultivo vertical y el estado del arte de su integración con la tecnología. Asimismo, se explican el objetivo y los alcances del trabajo.

### 1.1. Agricultura vertical como solución sostenible

Con una población mundial que supera los 8000 millones de personas y continúa en crecimiento [1], la agricultura enfrenta el desafío de volverse más eficiente y sostenible [2], [3]. En este contexto, los cultivos verticales emergen como una solución innovadora que optimiza el uso del espacio y los recursos, especialmente en entornos urbanos, donde el suelo es limitado y costoso [4].

Este tipo de agricultura utiliza técnicas como la hidroponía, que permite un uso preciso del agua, y la aeroponía, que maximiza el oxígeno disponible para las raíces. Además, las granjas verticales aprovechan áreas infrautilizadas, como edificios abandonados o naves industriales, y permiten cultivar alimentos en zonas donde la agricultura tradicional resulta difícil de desarrollar. Así, se logra una mayor densidad de cultivo y se contribuye a la sostenibilidad al reducir el uso de pesticidas y fertilizantes.

Aunque los cultivos verticales requieren un alto nivel de tecnología y tienen costes energéticos asociados, su capacidad para ahorrar recursos, disminuir la huella de carbono y fomentar la producción de alimentos en espacios reducidos y con menor dependencia de factores climáticos, justifica la inversión en su desarrollo y expansión [5].

Este método, también conocido como agricultura urbana, permite el crecimiento de plantas en interiores sin necesidad de luz solar directa, gracias a la aplicación de tecnologías agrícolas avanzadas. Estas optimizan las condiciones de cultivo, lo que facilita la propagación de plantas jóvenes y la producción de alimentos más saludables sin el uso de pesticidas.

Además, maximizan la producción al emplear sistemas de iluminación especializados de nueva generación, diseñados para estructuras multicapa. Gracias a esta tecnología, es posible obtener mayores rendimientos en un espacio reducido, consolidando a la agricultura vertical como una solución sostenible y eficiente para la producción de alimentos en entornos urbanos y con recursos limitados [4].

### 1.1.1. Tipos de cultivos verticales

Existen distintos tipos de agricultura vertical, que optimizan el crecimiento de las plantas sin grandes extensiones de suelo y emplean estructuras especializadas para maximizar la producción. A continuación, se describen los principales tipos de cultivos verticales y sus características.

#### Hidroponía

Es un método de cultivo sin suelo que suministra nutrientes a las plantas a través de una solución acuosa, con raíces en un medio inerte como lana de roca o perlita. Es un sistema eficiente en el uso de agua, permite un control preciso de nutrientes y favorece una mayor densidad de cultivo y un crecimiento más rápido que los métodos tradicionales [4], [6].

#### Aeroponía

Es un método de cultivo en el que las raíces de las plantas quedan suspendidas en el aire y se rocían con una solución nutritiva. Maximiza la oxigenación, acelera el crecimiento y optimiza el uso de nutrientes, aunque requiere un control preciso para evitar la desecación de las raíces [4], [6].

#### Acuaponía

Combina la cría de peces con la hidroponía, utilizando los desechos de los peces como fertilizante para las plantas, mientras estas purifican el agua. Es un sistema sostenible que minimiza el desperdicio, promueve la biodiversidad y es viable en entornos urbanos y rurales [6].

### 1.1.2. Sistemas de agricultura vertical más conocidos

Los sistemas de agricultura vertical varían según el tipo de cultivo y la técnica más adecuada para cada necesidad. A continuación, se describen los principales sistemas de cultivos verticales y sus características.

#### Sistemas de torre

Las torres verticales permiten el cultivo de plantas en estructuras cilíndricas apiladas, con el fin de maximizar el uso del espacio y facilitar tanto la recolección como el mantenimiento. Este sistema es ideal para cultivos de hoja verde y hierbas, y se integra fácilmente en espacios urbanos como balcones y terrazas [6].

#### Sistemas en rack o estanterías

Estos sistemas emplean estanterías apiladas donde las plantas crecen en bandejas o contenedores, son ideales para invernaderos y ambientes controlados. Facilitan el riego, la recolección y el monitoreo, al mismo tiempo que se optimiza el uso del espacio. Los sistemas en rack o estanterías son especialmente útiles para cultivos que requieren diferentes niveles de luz y humedad [6].



### **Sistemas montados en pared**

En estos sistemas, las plantas crecen en módulos montados en paredes. Su integración en la arquitectura no solo optimiza el espacio, sino que también aporta beneficios estéticos y funcionales. Los jardines verticales pueden mejorar la calidad del aire y contribuir al aislamiento térmico de los edificios [6].

### **Sistemas de bastidor en A**

El bastidor en forma de "A" proporciona una estructura estable y accesible para el cultivo vertical, lo que a su vez facilita el acceso a la luz y el riego. Este diseño es particularmente útil para cultivos que necesitan un soporte adicional, como tomates y pepinos, y puede ser utilizado tanto en interiores como en exteriores [6].

### **Sistemas de contenedores**

Estos sistemas utilizan contenedores apilables que pueden moverse y configurarse según las necesidades, adaptándose a diversos entornos y tipos de cultivo. Los contenedores son ideales para cultivos modulares y pueden ser personalizados para optimizar el uso del espacio y los recursos [6].

## **1.2. Motivación**

Este trabajo surge como un desarrollo de interés personal, impulsado por el deseo de contribuir al desarrollo sustentable y la optimización de los recursos naturales en la producción de alimentos. En la actualidad, el crecimiento de la población mundial y el aumento en la demanda de productos agrícolas presentan grandes desafíos ambientales y económicos.

La producción tradicional de alimentos enfrenta dificultades debido a la explotación intensiva de suelos, el uso excesivo de fertilizantes y pesticidas y el desperdicio de recursos esenciales como el agua. Con el fin de maximizar sus ganancias, muchas empresas recurren a métodos poco sostenibles que generan un impacto ambiental negativo, y afectan la biodiversidad, la calidad del suelo y el bienestar de las comunidades locales que dependen de estas fuentes de producción.

Uno de los problemas más críticos en la agricultura tradicional es el significativo desperdicio de agua en el riego. Se estima que se desperdicia hasta el 84 % del agua utilizada para riego [7]. Esta pérdida ocurre principalmente por evaporación, drenaje o absorción ineficiente. En un contexto donde la crisis hídrica es una amenaza global, desarrollar sistemas más sostenibles se vuelve una necesidad urgente.

Por otro lado, existe una creciente preocupación entre los consumidores sobre la calidad y procedencia de los alimentos. Cada vez más personas optan por un estilo de vida saludable, adoptando hábitos de consumo responsables, como el vegetarianismo, el veganismo y el autocultivo. Sin embargo, muchos de estos consumidores no cuentan con espacios adecuados ni conocimientos técnicos para producir sus propios alimentos, lo que genera una gran oportunidad para soluciones tecnológicas que faciliten la agricultura doméstica.

A partir de estas problemáticas, nació la motivación para desarrollar un sistema hidropónico inteligente, que no solo optimice el uso del agua, sino que también permita una gestión eficiente del cultivo con mínima intervención del usuario. Este trabajo ofrece una alternativa viable y escalable para la producción de alimentos frescos, reduce la dependencia de sistemas agrícolas tradicionales y promueve un modelo más sustentable para el futuro.

### 1.3. Estado del arte

Durante la etapa de investigación, se llevó a cabo una búsqueda de productos comerciales tanto en el mercado local como en el internacional. Se identificaron diversas soluciones con características similares al sistema desarrollado.

A continuación, se describen los productos identificados.

#### NIDO PRO

Como se muestra en la figura 1.1, el NIDO PRO es un sistema hidropónico inteligente para cultivos verticales que permite controlar la solución nutritiva a través de una aplicación móvil. Facilita la gestión del pH y la conductividad eléctrica (CE), con algoritmos que realizan comprobaciones diarias para garantizar la estabilidad de la solución nutritiva. Además, admite hasta cuatro ranuras para fertilizantes líquidos y ajustes de pH, lo que optimiza el proceso de nutrición de las plantas [8].



FIGURA 1.1. Ilustración del módulo NIDO PRO conectado a un sistema hidropónico horizontal<sup>1</sup>.

#### Xiaomi Mi Flower Care Plant Sensor

El Xiaomi Mi Flower Care Plant Sensor, que se muestra en la figura 1.2, monitorea variables ambientales como luz, humedad, nutrientes y temperatura del sustrato. Los sensores se conectan a aplicaciones móviles, lo que permite realizar un seguimiento detallado y controlar las condiciones del cultivo de manera precisa [9].

<sup>1</sup>Imagen tomada de <https://acortar.link/uEmExI>.



FIGURA 1.2. Ilustración del dispositivo Xiaomi Mi Flower Care Plant Sensor y sus prestaciones<sup>2</sup>.

### Kit de Riego Hydro de Konyks

El sistema de riego inteligente Konyks, que se muestra en la figura 1.3, permite controlar la llave de paso de forma remota mediante una aplicación móvil y asistentes de voz como Alexa, Google Home o Siri. Incorpora un medidor de caudal para monitorear el consumo de agua y una toma con relé RF que amplía la conectividad hasta 60 m y permite gestionar hasta cuatro grifos. Además, ofrece automatización programable según el clima y los horarios, lo que optimiza el uso del agua y facilita la gestión del riego desde cualquier lugar [10].



FIGURA 1.3. Sistema de riego conectado Konyks<sup>3</sup>.

<sup>2</sup>Imagen tomada de <https://acortar.link/bVNwHC>.

<sup>3</sup>Imagen tomada de <https://konyks.com/es/producto/hydro-kit/>.

### Smart 9 Pro de Click & Grow

El Smart Garden 9 PRO, que se muestra en la figura 1.4, es un sistema de cultivo doméstico automatizado con control a través de una aplicación. Ofrece riego automático, iluminación ajustable tanto por control táctil como desde la app, y un suministro equilibrado de nutrientes y oxígeno en las raíces. Permite cultivar hierbas, frutas, ensaladas y flores durante todo el año, con la opción de utilizar cápsulas presembradas (más de 50 variedades disponibles) o semillas propias. Incluye un set inicial con cápsulas de tomate, albahaca y lechuga [11].

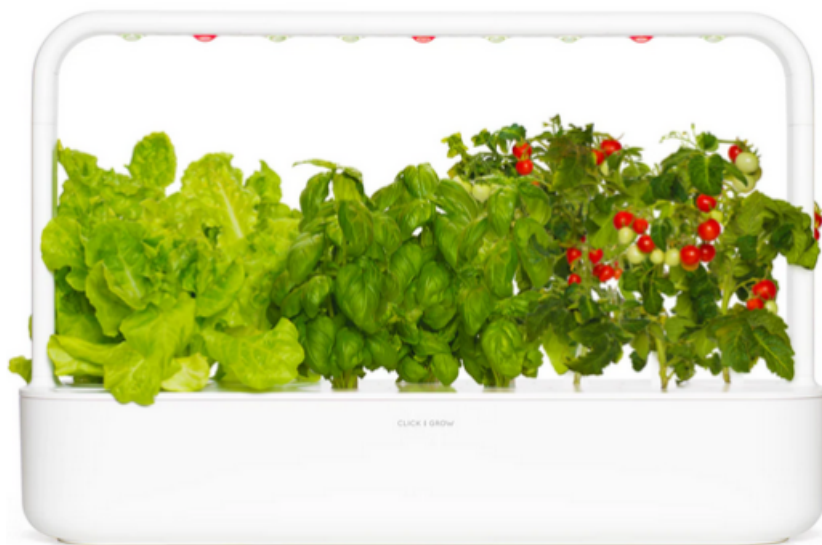


FIGURA 1.4. Módulo Smart Garden 9 PRO<sup>4</sup>.

## 1.4. Objetivos y alcances

A continuación, se presentan los objetivos que se plantearon para el desarrollo de este trabajo.

### 1.4.1. Objetivos

- Desarrollar un prototipo de sistema hidropónico automatizado para monitoreo y gestión eficiente del cultivo.
- Implementar la conexión de sensores para medir temperatura, humedad, pH, conductividad eléctrica, nivel de agua e intensidad lumínica.
- Diseñar una interfaz web y móvil que permitiera la supervisión y configuración remota del sistema.
- Evaluar el rendimiento del sistema en comparación con métodos tradicionales, midiendo eficiencia y productividad.
- Poner en práctica los conocimientos adquiridos a lo largo de la carrera de especialización en sistemas embebidos.

---

<sup>4</sup>Imagen tomada de <https://www.clickandgrow.com/products/the-smart-garden-9-pro>.

### **1.4.2. Alcances**

A continuación, se presentan los alcances que se lograron en este trabajo:

- El sistema permite el monitoreo en tiempo real y la gestión automatizada del riego, iluminación y ventilación.
- El dispositivo se enfocó en cultivos hidropónicos verticales de pequeña y mediana escala.
- Se realizaron pruebas de funcionamiento y validación en un entorno controlado, pero no se contempló una implementación comercial en esta etapa.



## Capítulo 2

# Introducción específica

En el presente capítulo se describen los componentes de hardware, software, protocolos de comunicación y plataformas utilizados para realizar el trabajo.

### 2.1. Componentes principales del hardware

En esta sección se describen los módulos de hardware de terceros utilizados en el trabajo.

#### 2.1.1. ESP32-DevKitC

La placa ESP32-DevKitC V4, que se muestra en la figura 2.1, es una placa de desarrollo basada en el SoC (*System On Chip*) ESP32 de Espressif Systems. Es una placa popular y versátil ampliamente utilizada por desarrolladores, ingenieros y aficionados para la creación de prototipos y el desarrollo de proyectos de internet de las cosas o IoT (del inglés, *Internet of Things*), sistemas embebidos y otras aplicaciones [12].

Características generales:

- Microcontrolador con arquitectura de uno o dos núcleos de 32 bits con velocidades de reloj de hasta 240 MHz.
- Memoria SRAM integrada.
- Conectividad Wi-Fi 802.11 b/g/n (2,4 GHz) integrada.
- Conectividad bluetooth (classic y low energy) integrada.

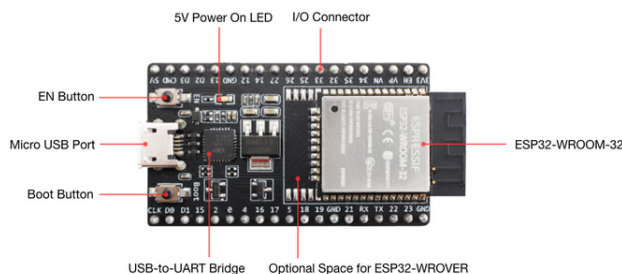


FIGURA 2.1. ESP32-DevKitC V4 con el módulo ESP32-WROOM-32 integrado<sup>1</sup>.

<sup>1</sup>Imagen tomada de <https://docs.espressif.com>.

### 2.1.2. Módulo sensor PH-4502C

El sensor de pH analógico PH-4502C, que se muestra en la figura 2.2, es un módulo electrónico diseñado para medir el grado de acidez de soluciones líquidas, típicamente el modelo E201-BNC [13]. Este sensor proporciona una salida de tensión analógica que es proporcional al nivel de pH detectado por el electrodo.

Características:

- Rango de detección de pH de 0 a 14.
- Salida analógica que varía, entre 0 y 5 VDC, con el pH del líquido.
- Temperatura de operación del módulo generalmente entre -10 °C y 50 °C.

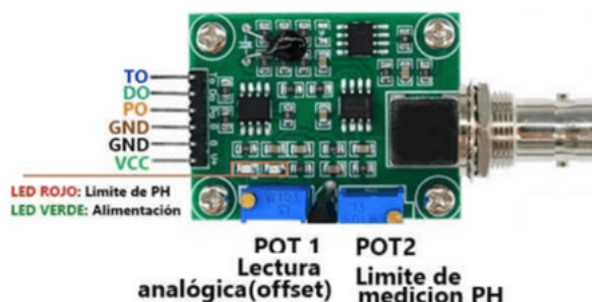


FIGURA 2.2. Distribución de pines del módulo PH-4502C<sup>2</sup>.

### 2.1.3. Módulo medidor de sólidos disueltos totales

El medidor de sólidos disueltos totales o TDS (del inglés, *Total Dissolved Solids*) Meter 1.0, que se muestra en la figura 2.3, es un sensor electrónico diseñado para medir la cantidad total de sustancias orgánicas e inorgánicas disueltas en un líquido, lo que se expresa típicamente en partes por millón (ppm) o miligramos por litro (mg/l) [14].

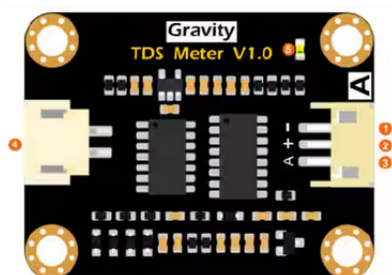


FIGURA 2.3. Ilustración del módulo sensor TDS meter v1.0<sup>3</sup>.

Este sensor se emplea para evaluar la calidad del agua en aplicaciones como hidroponía, acuicultura, tratamiento de aguas y monitoreo ambiental.

Características:

- Salida analógica de 0 a 2,3 VDC (proporcional a TDS).

<sup>2</sup>Imagen tomada de <https://uelectronics.com/producto/sensor-de-ph-liquido/>.

<sup>3</sup>Imagen tomada de <https://www.digikey.be/htmldatasheets>.



- Rango de medición típico de 0 a 1000 ppm.
- Interfaz de 3 pines (VCC, GND, output).
- Conector para la sonda (2 pines).

#### 2.1.4. Sensor de humedad capacitivo V2.0

El sensor de humedad capacitivo V2.0, que se muestra en la figura 2.4, mide los niveles de humedad mediante detección capacitiva en lugar de resistiva, como otros sensores disponibles. Fabricado con material resistente a la corrosión, ofrece una vida útil prolongada al insertarse en el sustrato alrededor de las plantas [15].

- Requiere alimentación entre 3,3 y 5,5 VDC.
- Corriente de operación de 5 mA.
- Salida analógica.



FIGURA 2.4. Módulo sensor de humedad capacitivo<sup>4</sup>.

#### 2.1.5. Sensor de temperatura digital DS18B20

El DS18B20, que se muestra en la figura 2.5, es un sensor de temperatura digital que proporciona mediciones en grados Celsius con una resolución configurable de 9 a 12 bits [16].

Características:

- Rango de medición de temperatura: -55 °C a +125 °C.
- Precisión:  $\pm 0,5$  °C en el rango de -10 °C a +85 °C.
- Interfaz de comunicación: 1-Wire (requiere un solo pin digital).

---

<sup>4</sup>Imagen tomada de <https://probots.co.in/>.



FIGURA 2.5. Pinout del sensor DS18B20 y presentación del chip con su vaina protectora característica<sup>5</sup>.

### 2.1.6. Sensor de luz ambiental digital BH1750

El circuito integrado BH1750, que se muestra en la figura 2.6, es un sensor de luz ambiental digital con interfaz de bus I2C (*Inter-Integrated Circuit*) [17]. Este sensor es capaz de detectar un amplio rango de intensidad luminosa, desde 1 hasta 65 535 lx, con una resolución de 16 bits. El chip proporciona una salida digital directa, lo que elimina la necesidad de cálculos complejos.

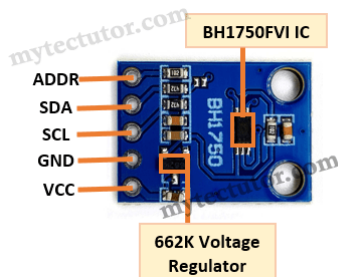


FIGURA 2.6. Pinout del módulo de adaptación del sensor BH1750<sup>6</sup>.

### 2.1.7. Raspberry Pi

Raspberry Pi es una plataforma de computación embebida basada en arquitectura ARM [18], capaz de ejecutar sistemas operativos Linux como Raspberry Pi OS [19]. Dispone de interfaces GPIO [20], I2C [21], SPI [22] y UART [23] que permiten la interacción directa con sensores, actuadores y periféricos externos. Su bajo consumo energético, soporte para redes Ethernet/Wi-Fi [24], [25] y capacidad de procesamiento la convierten en una herramienta versátil para el desarrollo, prueba e implementación de sistemas embebidos y aplicaciones IoT [19].

## 2.2. Componentes principales del software

En esta sección se describen las herramientas de software de terceros utilizados en el trabajo.

<sup>5</sup>Imagen tomada de <https://tienda.ityt.com.ar/sensor-temp-hum-ic/>.

<sup>6</sup>Imagen tomada de <https://mytectutor.com/>.

### 2.2.1. ESP-IDF

ESP-IDF (*Espressif IoT Development Framework*) es un conjunto de herramientas de desarrollo integral provisto por Espressif Systems. Este framework facilita la creación de firmware para su línea de SoCs ESP32 y ESP8266. Incluye un sistema operativo en tiempo real (FreeRTOS), bibliotecas con APIs para diversos periféricos y protocolos (Wi-Fi, bluetooth, TCP/IP), compilador (basado en GCC), depurador (GDB) y utilidades para la construcción, flasheo y monitoreo de proyectos. ESP-IDF permite a los desarrolladores escribir aplicaciones en C o C++, quienes aprovechan así la potencia y la conectividad de los chips de Espressif [26].

### 2.2.2. FreeRTOS

FreeRTOS es un sistema operativo en tiempo real (RTOS) popular y de código abierto. Ofrece un núcleo pequeño y eficiente, apropiado para microcontroladores y sistemas con recursos limitados. Proporciona mecanismos de multitarea, como hilos (tareas), gestión de memoria, sincronización y comunicación entre tareas (semáforos, mutexes, colas). Facilita la organización y la gestión de la ejecución de múltiples funciones de manera concurrente y determinista, crucial para aplicaciones de tiempo real. Su portabilidad permite su uso en una amplia variedad de arquitecturas de procesadores [27].

### 2.2.3. Ceedling

Ceedling es un framework de construcción y prueba para proyectos de software embebido en lenguaje de programación en C. Automatiza tareas como la compilación, el enlazado y la ejecución de pruebas unitarias. Integra herramientas como Unity, CMock y Ruby. Facilita la adopción de prácticas de desarrollo basadas en pruebas (TDD) y asegura la calidad del código mediante la verificación automatizada de unidades de software individuales [28].

### 2.2.4. React

React es una biblioteca de código abierto desarrollada por Meta para crear interfaces de usuario dinámicas en aplicaciones web. Su enfoque basado en componentes y el uso del DOM virtual permiten desarrollar interfaces modulares y con un alto rendimiento, ideales para aplicaciones modernas y escalables [29].

### 2.2.5. Docker

Docker es una plataforma de código abierto que permite empaquetar, distribuir y ejecutar aplicaciones en entornos aislados llamados contenedores. Estos garantizan que el software funcione de forma consistente en cualquier entorno, facilitando la portabilidad, la escalabilidad y la automatización del despliegue de aplicaciones [30].

## 2.3. Protocolos de comunicación empleados

A continuación, se detallan los protocolos de comunicación empleados en la realización del trabajo.

### 2.3.1. HTTP

HTTP (*Hypertext Transfer Protocol*) es un protocolo de aplicación que define cómo los clientes (navegadores web) solicitan recursos (páginas web, imágenes, etc.) a los servidores y cómo estos responden. Utiliza un modelo de petición-respuesta. Las peticiones HTTP incluyen un método (GET, POST, PUT, DELETE, etc.) que indica la acción que el cliente desea realizar. Las respuestas HTTP contienen un código de estado que informa sobre el resultado de la petición [31].

### 2.3.2. I2C

I2C es un protocolo de comunicación serial síncrono, multi-maestro/esclavo, de baja velocidad y corta distancia. Utiliza solo dos líneas bidireccionales: SDA (datos seriales) y SCL (reloj serial), ambas conectadas a través de resistencias *pull-up*. Permite que múltiples dispositivos se comuniquen entre sí en el mismo bus. Los maestros inician la comunicación y controlan el reloj, mientras que los esclavos responden a las peticiones de los maestros [21].

### 2.3.3. 1-Wire

1-Wire es un protocolo de comunicación serial semidúplex. Utiliza un único conductor para la comunicación de datos y, en algunos casos, para la alimentación. Un maestro controla la comunicación con uno o varios dispositivos esclavos en el mismo bus. El protocolo es relativamente lento pero resulta económico para conectar sensores, memorias y otros dispositivos de baja velocidad, especialmente en aplicaciones donde el bajo número de pines es limitado [32].

## Capítulo 3

# Diseño e implementación

En este capítulo se describe la arquitectura general del sistema, el diseño del software, su estructura lógica y los módulos que lo conforman. Además, se detalla el proceso de diseño del hardware, la selección y calibración de sensores, así como el desarrollo de la aplicación móvil.

### 3.1. Diagrama de bloques

En la figura 3.1 se muestra el diagrama en bloques general del sistema donde se describe la arquitectura aplicada al trabajo.

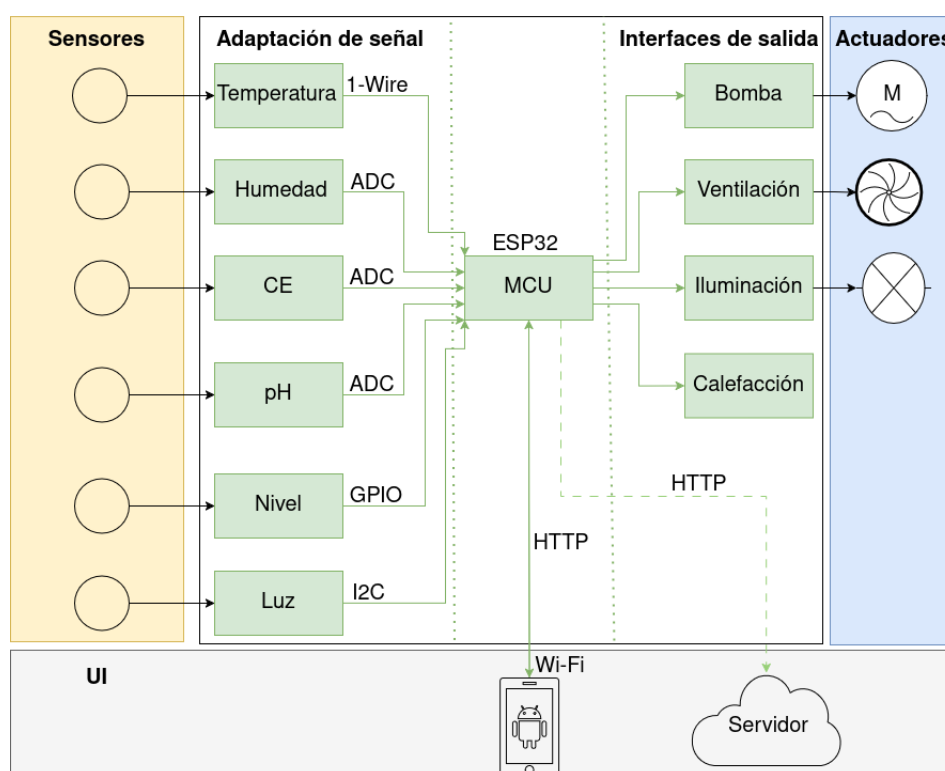


FIGURA 3.1. Diagrama de bloques del sistema.

El sistema embebido implementado en este trabajo consta de una PCB (del inglés, *Printed Circuit Board*) centralizadora, diseñada para integrar y gestionar todos los módulos de hardware. Esta arquitectura asegura la alimentación, adaptación y protección de todos sus componentes. El sistema completo abarca desde la adquisición de datos mediante sensores hasta las acciones sobre el entorno a través

de actuadores. Además, se complementa con una interfaz de usuario móvil para el monitoreo y control remoto.

A continuación, se describen brevemente los bloques y su función:

- **Sensores:** este bloque se encarga de la transducción de magnitudes físicas en señales eléctricas, lo que permite la digitalización de variables ambientales de interés para el control del cultivo.
- **Adaptación de señal:** los módulos de adaptación acondicionan las señales provenientes de los sensores. Para garantizar la correcta interpretación por parte de la unidad de microcontrolador o MCU (del inglés, *MicroController Unit*), ajustan los niveles de tensión y la relación señal-ruido a valores apropiados.
- **MCU:** el núcleo del sistema, basado en el chip ESP32, comanda la comunicación y el control de todos los módulos. Provee la capacidad de procesamiento y la conectividad inalámbrica necesarias para la automatización del cultivo y la interacción con la aplicación móvil.
- **Interfaces de salida:** proporcionan aislamiento galvánico y acondicionamiento de potencia para la activación de los actuadores, lo que asegura la protección del MCU y la correcta operación de los componentes de mayor potencia.
- **Actuadores:** los actuadores (ventiladores, bomba de irrigación, luces, resistencia calefactora) ejecutan las acciones de control y modifican las condiciones ambientales del cultivo según las necesidades.
- **Aplicación móvil de usuario:** desarrollada para facilitar la interacción con el sistema, la aplicación móvil permite el monitoreo en tiempo real de las condiciones del cultivo y el control remoto de los actuadores.
- **Interfaz de servicio web:** se implementó una interfaz para la transmisión de datos a un servicio web externo que permite el almacenamiento y análisis de información del cultivo. Esta funcionalidad se encuentra fuera del alcance principal de este trabajo.

## 3.2. Arquitectura del firmware

En la presente sección se aborda la arquitectura del firmware del microcontrolador.

### 3.2.1. Patrones

A continuación, se detallan los patrones de diseño arquitectónico utilizados.

#### Arquitectura en capas

Se adoptó un patrón de arquitectura en capas para estructurar el software desarrollado, lo que permite una separación de funcionalidades clara mediante niveles de abstracción. Dicha metodología divide el sistema en niveles horizontales, cada uno con responsabilidades específicas y bien definidas, que facilita el desarrollo, la mantenibilidad y la escalabilidad del código.

A continuación, se enumeran las capas de abstracción que constituyen el firmware:

- Capa de aplicación.
- Capa de sistema operativo.
- Capa de abstracción de hardware (HAL).

### Capa de abstracción de hardware

Para facilitar la interacción con los diversos componentes de hardware y garantizar la portabilidad del código, se implementó una capa de abstracción basada en Espressif HAL (*Hardware Abstraction Layer*). Integrada dentro del SDK de ESP-IDF, esta capa proporciona una interfaz uniforme para el control de los periféricos del ESP32, independientemente de las particularidades del hardware subyacente.

### Control ambiental

El patrón de control ambiental se adoptó como estrategia arquitectónica para la capa de aplicación. El sistema embebido requirió la monitorización y modificación del entorno mediante sensores y actuadores. Este patrón permite estructurar la lógica de control y facilita la gestión de las interacciones entre los componentes de hardware y la implementación de los algoritmos de control.

#### 3.2.2. Componentes

Como se muestra en la figura 3.2, la capa de aplicación está constituida por los componentes de software encargados de gestionar cada una de las funcionalidades del sistema.

A continuación, se describe brevemente la funcionalidad de cada uno de estos componentes:

- Monitor de temperatura: mide la temperatura de la solución nutritiva para garantizar que las plantas crezcan en un entorno térmicamente adecuado.
- Monitor de conductividad eléctrica (CE): evalúa la concentración de nutrientes presentes en la solución hidropónica.
- Monitor de nivel: monitorea que el nivel de solución nutritiva en el sistema no caiga por debajo de un valor determinado.
- Monitor de pH: mide el nivel de acidez de la solución nutritiva, lo que ayuda a mantener el pH dentro del rango óptimo para el cultivo.
- Monitor de humedad del sustrato: mide el contenido de humedad en el sustrato del cultivo.
- Monitor de luz: mide la cantidad de luz disponible en el entorno a través de un sensor.
- Servidor web embebido: permite la comunicación con el sistema de control del cultivo a través de la red. Este servidor permite monitorear, configurar y controlar el sistema desde cualquier dispositivo con acceso a la red.

- **Control de ciclo de luz:** este módulo se encarga de gestionar el ciclo de iluminación del cultivo. La configuración de dicho ciclo se realiza por medio de la interfaz de usuario.
- **Control de ciclo de oxigenación/ventilación:** este componente activa y desactiva el flujo de aire en el sistema de cultivo según la configuración del usuario.
- **Control de hidratación:** gestiona el riego, activa la bomba de agua y ajusta los niveles de humedad del sustrato según las mediciones del sensor de humedad. Esto asegura la cantidad adecuada de agua para el desarrollo de las plantas.
- **Administrador:** es el componente encargado de la centralización y validación global de los datos suministrados por los demás módulos de software.

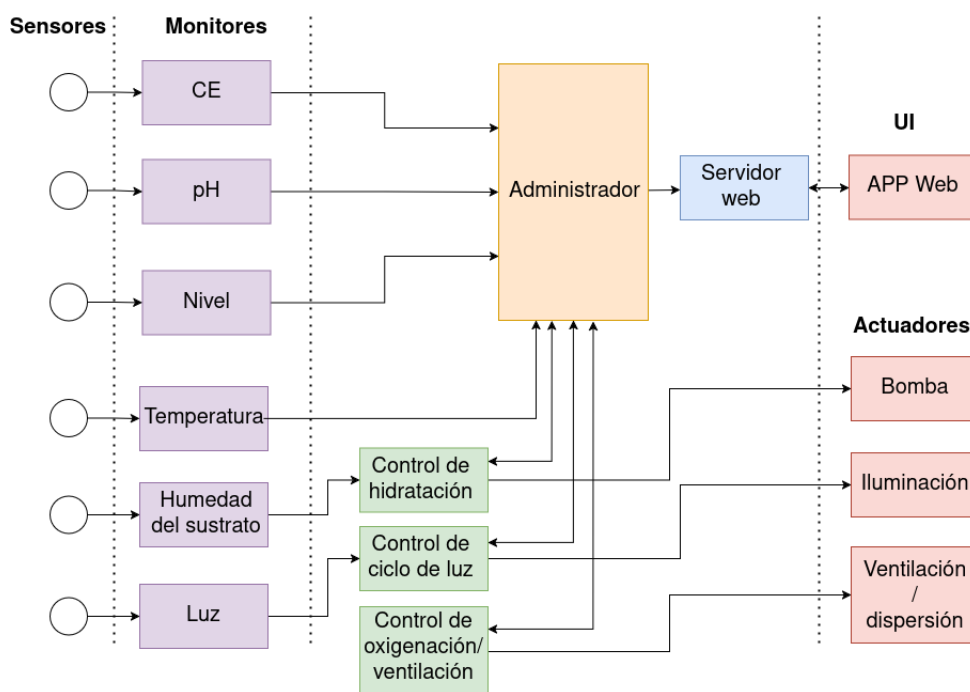


FIGURA 3.2. Diagrama de módulos funcionales y sus interacciones.

### 3.3. Desarrollo del software

Para facilitar la escalabilidad de la aplicación, se implementó una tarea de FreeRTOS para cada componente del firmware. Esta arquitectura modular permite modificar cada elemento de forma independiente, sin afectar al resto del sistema.

#### 3.3.1. Gestión de la comunicación entre tareas

La comunicación entre tareas se gestiona mediante un patrón de publicación/suscripción con múltiples canales, que utiliza colas como buffers para el intercambio de datos. Esta arquitectura desacopla los productores de información (monitores) de los consumidores (controladores y administrador), lo que garantiza la integridad de los datos y la estabilidad del sistema al evitar condiciones de carrera.



### 3.3.2. Gestión de prioridades

Se asignaron diferentes niveles de prioridad a las tareas según su criticidad. Las de monitoreo se configuraron con un nivel intermedio, mientras que las de control y administración recibieron mayor relevancia para asegurar tiempos de respuesta adecuados. El servidor, al depender de la interacción del usuario, ocupa la posición más elevada en la jerarquía.

Los monitores funcionan como tareas productoras que obtienen datos del entorno mediante sensores y los publican en estructuras compartidas. Los controladores actúan como tareas consumidoras que toman decisiones basadas en estos datos y accionan los actuadores correspondientes. La tarea de administrador, también consumidora, coordina el funcionamiento general del sistema y gestiona configuraciones.

### 3.3.3. Controladores de dispositivos

Se desarrollaron controladores de dispositivos (*device drivers*) dedicados a gestionar directamente las unidades de hardware, mediante la abstracción de las operaciones de bajo nivel necesarias para interactuar con sensores y actuadores. En el caso particular del ADC, se implementó una capa intermedia con el patrón singleton para la construcción de la biblioteca de acceso, con el fin de evitar condiciones de carrera durante el acceso al periférico compartido entre múltiples tareas.

### 3.3.4. Gestión de fallos

El sistema incorpora mecanismos de detección y manejo de fallos a nivel de hardware y software. Cada tarea cuenta con rutinas de supervisión que identifican comportamientos anómalos o pérdida de respuesta de los dispositivos.

En caso de error, se activan procedimientos de contingencia que buscan restablecer el funcionamiento sin alterar la estabilidad general. Por ejemplo, si un sensor deja de enviar datos válidos durante un tiempo determinado, se genera una alerta que es registrada y comunicada al usuario. Si un actuador no responde tras múltiples intentos, se desactiva su uso y se notifica mediante una bandera de estado.

Además, se implementó un mecanismo de *watchdog* que reinicia automáticamente el microcontrolador en caso de bloqueo o error crítico no recuperable. La configuración del usuario se almacena en memoria no volátil (NVS) por medio de un sistema de archivos, lo que permite la recuperación de parámetros tras cortes de energía.

### 3.3.5. Servidor embebido y API REST

El sistema cuenta con un servidor embebido HTTP alojado en la ESP32 que expone una API REST para la interacción con el usuario. Esta permite el monitoreo del estado del cultivo, la consulta de valores de sensores en tiempo real, y la ejecución de acciones como el encendido manual de actuadores o la configuración del dispositivo.

El servidor opera concurrentemente con el resto de las tareas gracias a la arquitectura multitarea de FreeRTOS, y cuenta con un sistema básico de autenticación por usuario y contraseña.

La API REST atiende peticiones HTTP desde navegadores y la aplicación PWA (*Progressive Web Application*) desarrollada en React, que se distribuye mediante un servidor Docker desplegado en una Raspberry Pi 4 Model B.

Las operaciones principales incluyen:

- Lectura de datos ambientales (temperatura, humedad, pH, EC).
- Consulta del estado de actuadores (iluminación, bomba, ventilación).
- Configuración de parámetros y umbrales de alerta.
- Activación manual o automática de funciones del sistema.
- Acceso a registros de eventos y errores.

El firmware mantiene una estructura centralizada de datos que se sincroniza con la aplicación móvil a través de sesiones HTTP válidas, lo que asegura que el usuario visualice información actualizada del cultivo.

### 3.3.6. Lógica de negocio

Al encender el sistema por primera vez, se configura automáticamente el acceso Wi-Fi mediante el protocolo WPS (*Wi-Fi Protected Setup*). El dispositivo espera la activación del botón WPS del router para obtener las credenciales, que se almacenan localmente para futuros reinicios. Si la conexión falla, el proceso se reinicia automáticamente.

Una vez establecida la conexión, se inicia el servidor embebido y se habilita la interacción con la aplicación móvil.

### 3.3.7. Estructura del código

La arquitectura sigue el patrón modular con módulos de software que ejecutan tareas de forma ordenada. El ciclo comienza en `main.c` con la inicialización del sistema a través del componente `Administrator`.

#### Inicialización del sistema

Como se ilustra en la figura 3.3, la función `app_main()` crea la tarea `administrator_callback` con alta prioridad, que actúa como un coordinador general que gestiona la interacción entre módulos y subsistemas.

Durante la inicialización, el `Administrator` configura el pool de memoria, inicializa el sistema de archivos, establece la conectividad Wi-Fi e inicia el servidor web.

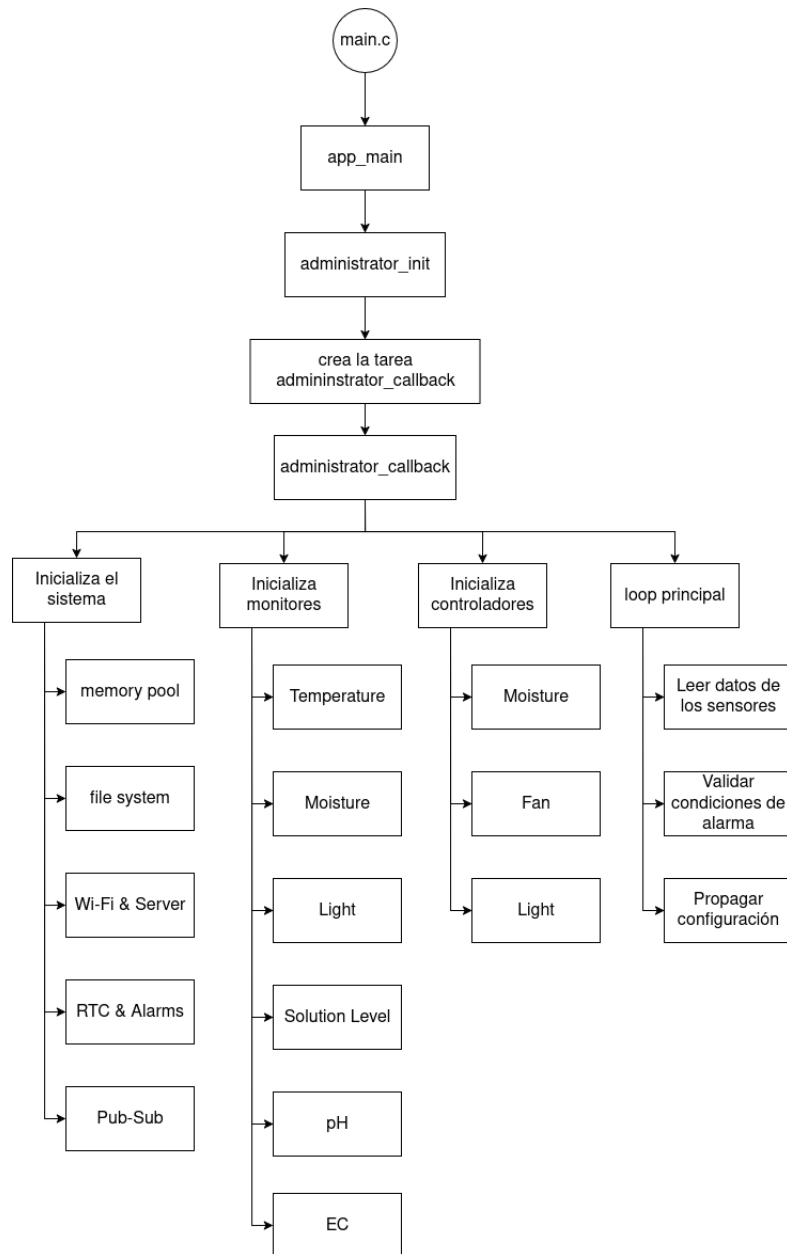


FIGURA 3.3. Diagrama simplificado del ciclo de ejecución del programa.

### Configuración de hora y alarmas

El sistema incorpora un módulo RTC (*Real Time Clock*) que garantiza precisión temporal durante interrupciones de energía. Además, inicializa un sistema de alarmas con niveles de severidad variables y persiste los mensajes en archivo para evitar pérdida de información tras reinicios.

### Creación de monitores de sensores

Se crean seis tareas independientes de monitoreo para sensores específicos del sistema hidropónico, gestionadas por sus respectivos *device drivers*:

- `temperature_monitor`: muestrea la temperatura (DS18B20).

- `moisture_monitor`: mide la humedad del sustrato (sensor capacitivo v2.0).
- `light_monitor`: muestrea la iluminancia (BH1750).
- `solution_level_monitor`: sensor de nivel de solución.
- `pH_monitor`: mide pH (módulo PH-4502C).
- `EC_monitor`: mide conductividad eléctrica y concentración de nutrientes.

Cada monitor es una tarea independiente que realiza lecturas cada segundo y publica datos hacia el administrador y el controlador correspondiente.

### Inicialización de controladores de actuadores

Paralelamente, se crean tres controladores como tareas independientes:

- `moisture_controller`: administra riego automático por nivel o intervalo.
- `light_controller`: controla luces según ciclo diurno para interiores.
- `fan_controller`: regula ventilación y circulación de aire.

Estos controladores operan de forma autónoma según la configuración establecida previamente por el usuario, que también puede enviar órdenes directas para activar o desactivar los actuadores.

### Mecanismo de comunicación Pub/Sub

El sistema utiliza un mecanismo de *publish/subscribe* (`pub_sub.h`) que permite interacción desacoplada entre componentes. Los sensores publican mediciones en tópicos específicos, mientras los controladores se suscriben a comandos de actuación. El `Administrator` supervisa el flujo completo de información.

### Bucle principal de control

Una vez inicializado, el `Administrator` ejecuta un bucle principal que se repite cada segundo y realiza las funciones detalladas en la figura 3.3:

- Lee datos publicados por sensores.
- Evalúa condiciones de alarma mediante comparación con umbrales configurados.
- Actualiza la API REST con información en tiempo real.
- Procesa modificaciones de configuración del usuario.

### Arquitectura de tareas paralelas

El diseño basado en tareas paralelas permite a sensores y controladores operar independientemente bajo supervisión del `Administrator`, como se ilustra en la figura 3.3. Esta estructura evita cuellos de botella y asegura respuesta rápida, lo que resulta en un programa robusto, escalable y mantenible.

### 3.3.8. Gestion de la configuración

Al iniciar, el sistema crea un archivo `config.json` en la memoria flash. Este archivo se genera con valores por defecto, ya que el usuario aún no ha configurado el dispositivo. Para su creación y almacenamiento se utilizan las bibliotecas `json_static.h` y `fs.h`, respectivamente. Simultáneamente, se inicializa en memoria RAM la estructura `g_current_config`, que mantiene la configuración durante toda la ejecución del programa. Esta estructura solo se actualiza mediante la interacción del usuario a través de la API REST, al utilizar el endpoint `/config` con el método POST.

Cuando se recibe una nueva configuración, se actualiza la variable global correspondiente definida en `hydro_rest.c`. Inmediatamente, se envía un mensaje a través del tópico `u_config` con un indicador que especifica qué módulo fue actualizado. Esto permite al administrador actualizar `config.json` y propagar la configuración al controlador correspondiente.

A continuación, en la figura 3.4 se puede observar un diagrama de flujo simplificado de la propagación de la configuración.

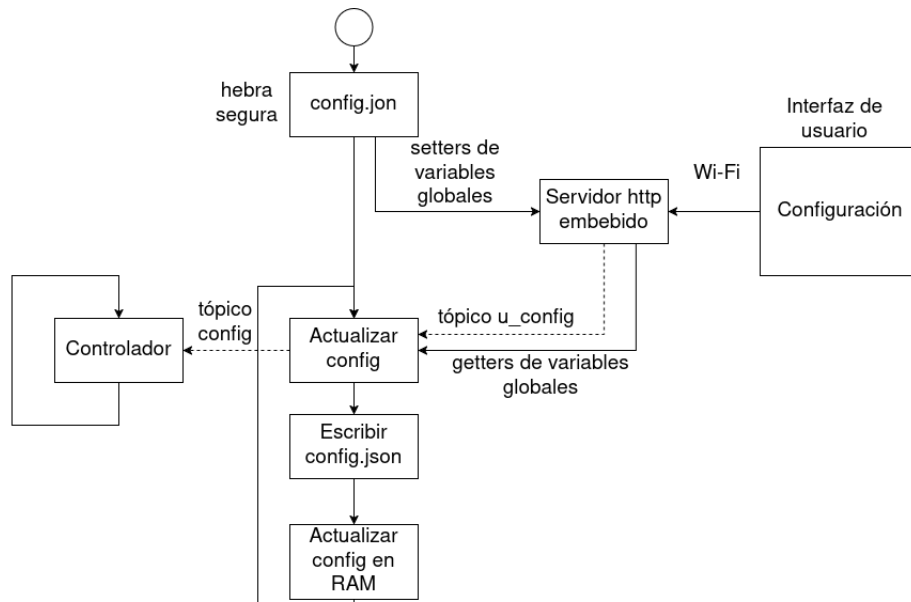


FIGURA 3.4. Propagación de la configuración.

## 3.4. Desarrollo del hardware

Esta sección describe el proceso completo de diseño, implementación y validación del hardware. Se abordan las etapas de planificación del circuito, selección de componentes específicos, integración de actuadores controlados, diseño de PCB y pruebas de funcionalidad.

### 3.4.1. Arquitectura general

El sistema está conformado por una placa principal que integra módulos independientes que trabajan de manera coordinada. Entre estos se incluyen el módulo de control principal basado en ESP32, los sensores ambientales, los actuadores y la alimentación.

### 3.4.2. Diseño electrónico

El diseño electrónico se realizó en base a una arquitectura modular centralizada en una placa principal. Esta placa interconecta los adaptadores de señal de los módulos sensores junto con la distribución de la alimentación, la conexión y protección de las GPIOs del microcontrolador junto con la protección general del circuito. La placa tiene completamente integrada la interfaz de salida con circuitos de aislamiento galvánico y relés para cada actuador, que manejan 220 V de corriente alterna. Para la alimentación de los módulos se implementó un regulador lineal LM1117, y para adaptar el módulo de pH de 5 V a 3,3 V (nivel de referencia del ADC) se utilizó un divisor resistivo. Finalmente, se incorporó un *shield* de salida con las GPIOs restantes para permitir la conexión de módulos adicionales en el futuro.

#### Módulo de control principal

El núcleo del sistema está basado en el microcontrolador ESP32-WROOM-32 montado en una placa de desarrollo ESP32-DevKitC, que proporciona conectividad Wi-Fi, múltiples interfaces de comunicación (1-Wire para el sensor DS18B20, I2C para el sensor BH1750, entradas analógicas para el sensor capacitivo, PH-4502C y conductividad eléctrica, entrada digital para el sensor de nivel) y capacidad de procesamiento para la ejecución de tareas en tiempo real.

#### Sensores

El sistema integra los sensores ambientales descritos en la sección 2.1. Desde la perspectiva de hardware, cada sensor se conecta mediante el protocolo apropiado a las interfaces del MCU. Las señales analógicas son acondicionadas mediante circuitos de adaptación. Los sensores digitales utilizan los protocolos nativos de comunicación para su integración.

#### Actuadores

Los actuadores fueron integrados para el control automatizado de las condiciones ambientales del cultivo hidropónico. El sistema incluye los siguientes actuadores:

- **Bomba de riego:** bomba sumergible para acuario de 220 V con caudal fijo (1 l/min).
- **Luces de crecimiento:** módulos LED full-spectrum (400-700 nm) con potencia de 20-50 W.
- **Ventiladores:** kit de ventiladores axiales de 12 V de caudal fijo con alimentación de 220 V.

El control de los actuadores se implementa mediante el sistema de relés mecánicos descrito en la sección 3.4.2, con interfaz a través de las salidas GPIO.

El diseño del PCB se realizó en KiCad. La figura 3.5 muestra el acondicionamiento de señal para los módulos de TDS y pH, junto con los circuitos de protección y regulación de tensión. La figura 3.6 muestra la interfaz de salida para la bomba de agua. Los demás actuadores emplean un circuito similar. Ambos esquemas forman parte de la placa centralizadora.

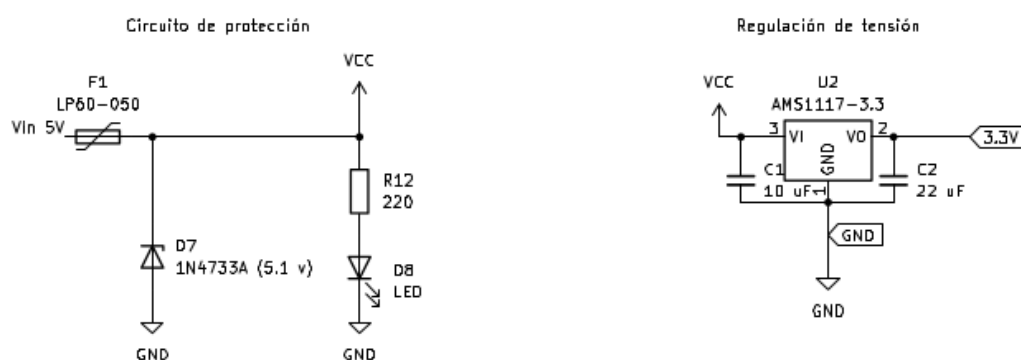
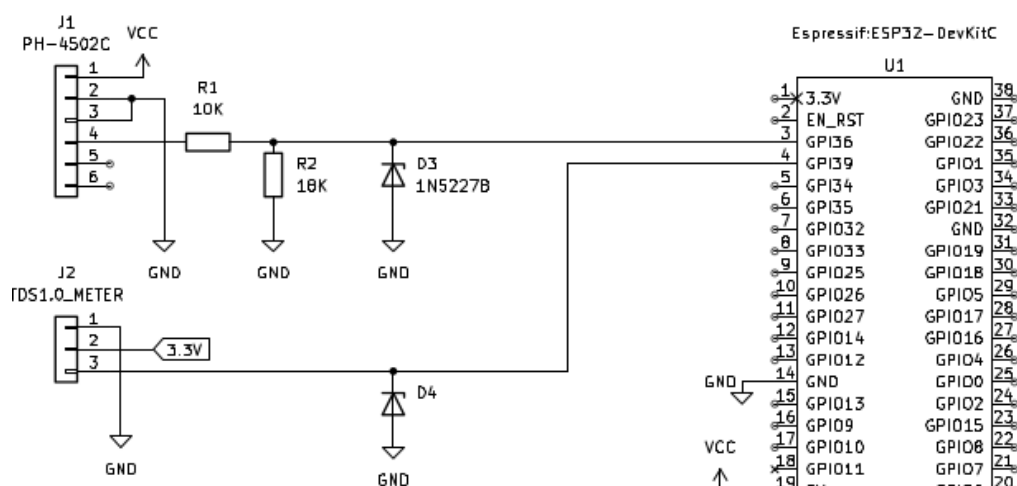


FIGURA 3.5. Vista parcial del circuito de la placa centralizadora que muestra la interconexión de los módulos de TDS y pH con el microcontrolador y el circuito de acondicionamiento de señal.

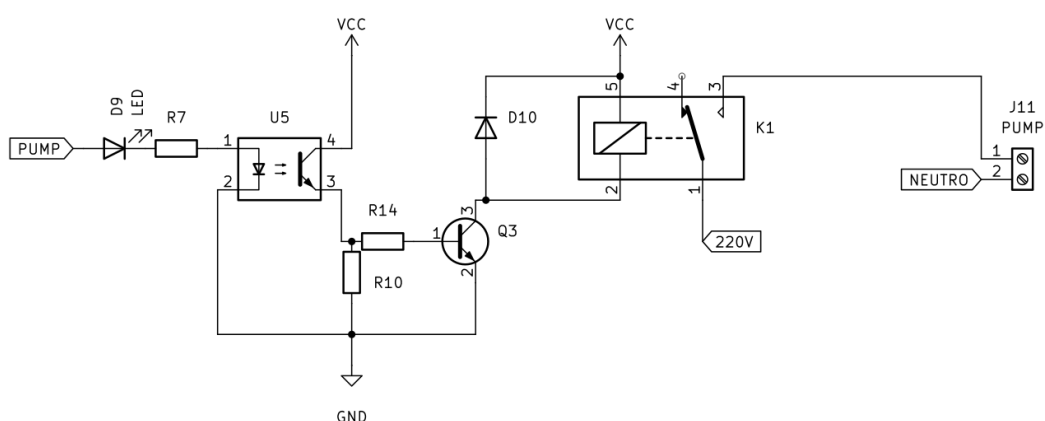


FIGURA 3.6. Circuito de control de la bomba con aislamiento galvánico y relé de potencia.

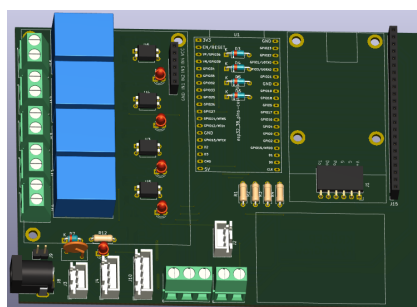
### 3.4.3. Alimentación

El circuito se alimenta con una fuente conmutada de 5 V como entrada principal. A partir de esta se generan los niveles de tensión requeridos: 3,3 V para la alimentación de sensores y módulos periféricos mediante reguladores lineales, y 5 V para el microcontrolador ESP32 y demás circuitos digitales. Se implementaron varias capas de protección que incluyen: fusibles de corriente nominal, diodos de polaridad inversa para prevenir daños por conexión errónea.

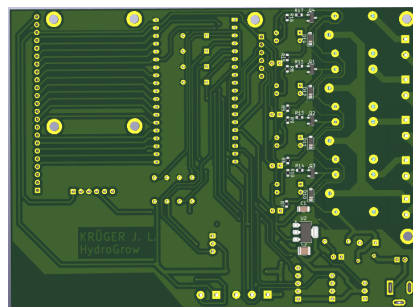
### 3.4.4. Diseño del PCB y ensamblado

El diseño del PCB se basa en principios prácticos de diseño electrónico. Se implementó un stack de dos capas con rutas de potencia y señales de control. Las pistas de alimentación se diseñaron con anchos de entre 1,0 mm y 2,0 mm para corrientes de hasta 4 A, mientras que las señales digitales utilizan pistas de 0,25 mm. Se incorporaron planos de masa continuos para mejorar la estabilidad del sistema.

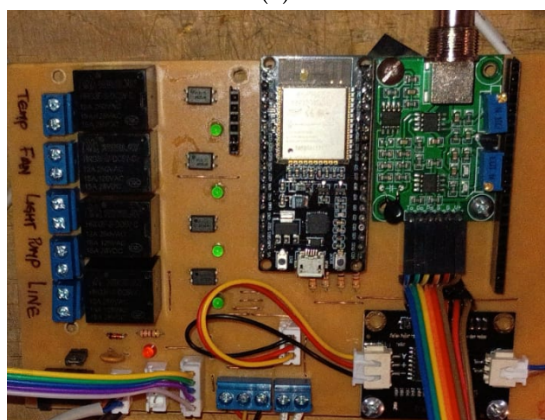
La disposición de componentes sigue una estrategia de flujo de señal lógico, con el microcontrolador ubicado en el centro y los conectores de sensores en los bordes para facilitar el cableado, como se puede observar en la figura 3.7. Los reguladores de tensión se colocaron cerca de sus cargas correspondientes para minimizar la caída de potencial en las pistas. Los capacitores de desacoplo se ubicaron adyacentes a cada pin de alimentación del microcontrolador para filtrar ruido de alta frecuencia.



(A) Vista 3D de KiCad. Lado frontal.



(B) Vista 3D de KiCad. Lado inferior.



(C) Vista final del PCB ensamblado.

FIGURA 3.7. Vistas del circuito impreso (PCB) del sistema.



El ensamblado se realizó de manera manual en un entorno controlado con temperatura y humedad estables. Todos los componentes se soldaron manualmente con cautín utilizando soldadura con plomo (60/40). Se aplicaron medidas de protección ESD durante todo el proceso y se realizó inspección visual y con lupa para verificar la calidad de las uniones soldadas.

#### **3.4.5. Consideraciones de seguridad y mantenimiento**

El diseño incluye aislamiento galvánico en las líneas de control de los actuadores, protecciones contra sobrecorriente y puntos de fácil acceso para mantenimiento. Estas medidas aumentan la durabilidad del sistema y reducen el riesgo de fallos eléctricos.

#### **3.4.6. Ensamblado final**

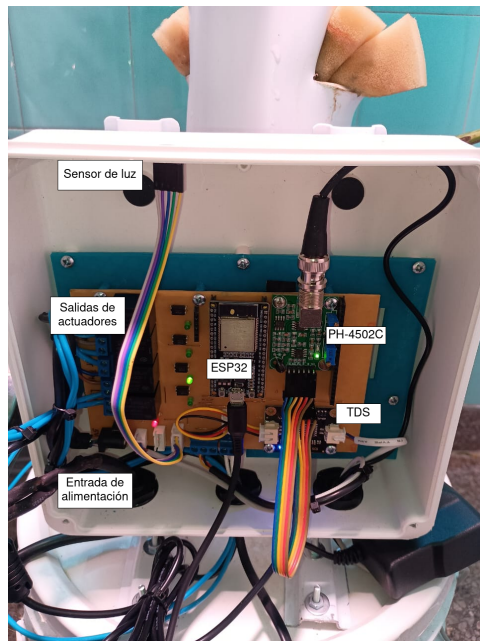
Una vez completado el montaje del PCB, se procedió a la integración de los sensores y actuadores al sistema principal. Los sensores DS18B20, capacitivo v2.0, BH1750, PH-4502C, conductividad eléctrica y nivel de solución se conectaron a los puertos correspondientes de la PCB mediante cables de señal y cables de alimentación. Los actuadores (bomba sumergible de 220 V, módulos LED full-spectrum y ventiladores axiales) se interconectaron al sistema de relés del PCB para su control automatizado.

La fuente de alimentación conmutada de 5 V se conectó al circuito de entrada, lo que proporciona la energía necesaria para todo el sistema. Para garantizar la protección contra condiciones ambientales adversas, la placa se montó dentro de un gabinete estanco con certificación IP65. Se verificaron todas las conexiones eléctricas y mecánicas antes de proceder con las pruebas funcionales. Luego, se realizó una inspección visual completa del ensamblado para asegurar la correcta integración de todos los componentes y la ausencia de conexiones sueltas o daños en el cableado.

Posteriormente, el sistema se instaló en el cultivo hidropónico. Los sensores se posicionaron estratégicamente en el sistema: el DS18B20 se sumergió en la solución nutritiva para monitoreo de temperatura, el sensor capacitivo se instaló en el sustrato para medir humedad, el BH1750 se ubicó sobre el gabinete para controlar iluminación, el PH-4502C y sensor de conductividad se colocaron en el depósito de solución y el sensor de nivel se fijó en el tanque de reserva.

Los actuadores se integraron al sistema hidropónico de la siguiente manera: la bomba sumergible se instaló en el depósito para recirculación, los módulos LED se montaron sobre el área de cultivo para iluminación, y los ventiladores se colocaron para circulación de aire. Finalmente, se procedió a la configuración de parámetros del sistema antes de iniciar las operaciones automatizadas.

En la figura 3.8 se muestra el montaje final del sistema.



(A) Montaje final del sistema con gabinete abierto.



(B) Montaje final del sistema con gabinete cerrado.

FIGURA 3.8. Montaje final del sistema.

### 3.5. Selección y calibración de sensores

Esta sección describe los criterios de selección y el proceso de calibración de los sensores utilizados. El objetivo es garantizar mediciones precisas y confiables de las variables de interés del entorno hidropónico.

#### 3.5.1. Selección de sensores

La selección de los sensores se realizó considerando precisión, estabilidad, compatibilidad eléctrica, facilidad de integración con el microcontrolador ESP32, disponibilidad en el mercado local y costo. Los sensores seleccionados fueron:

- Temperatura: sensor digital DS18B20, con rango de medición de  $-55\text{ }^{\circ}\text{C}$  a  $+125\text{ }^{\circ}\text{C}$  y resolución de 12 bits. El rango excede ampliamente los requisitos de  $-15\text{ }^{\circ}\text{C}$  a  $50\text{ }^{\circ}\text{C}$  del sistema, con una precisión de  $\pm 0,0625\text{ }^{\circ}\text{C}$  que supera el  $\pm 1\text{ }^{\circ}\text{C}$  requerido, lo que asegura mediciones confiables en condiciones ambientales variables.
- Humedad del sustrato: sensor capacitivo analógico, resistente a la corrosión y con respuesta lineal en el rango de 0-100 %. Cumple exactamente con el rango requerido de 0-100 % y la precisión de  $\pm 3\text{ }%$ , con respuesta lineal que facilita la calibración y asegura mediciones estables en ambientes húmedos y corrosivos.
- pH: sonda analógica con amplificador de alta impedancia, rango de 0-14, calibrable mediante buffer estándar de pH 4, 7 y 10. El rango amplio (0-14) cubre holgadamente el intervalo requerido de 5,5-7,5, con calibración estándar que garantiza la precisión de  $\pm 0,1$  especificada para el monitoreo de soluciones nutritivas.

- Conductividad eléctrica (EC): sensor TDS Meter v1.0 tipo electrodo doble, con rango de medición de 0-1000 ppm (equivalente a 0-2 mS/cm). Aunque el rango de 0-2 mS/cm no cubre completamente el requisito especificado de 1-4 mS/cm, sí abarca el intervalo necesario para cultivos hidropónicos (0-2 mS/cm), con las ventajas adicionales de su bajo costo y amplia disponibilidad tanto en el mercado local como internacional.
- Nivel de solución: sensor de flotador con interruptor convencional y resistente al ambiente húmedo. Proporciona una detección simple y confiable del nivel mínimo requerido (3/4 de la altura del contenedor) mediante una señal digital clara, ideal para alertas automáticas sin necesidad de mediciones continuas.
- Luminosidad: sensor digital BH1750 con rango de 1 a 65 535 lx. El amplio rango dinámico permite una medición precisa de la iluminación artificial requerida para el ciclo día-noche del cultivo, con interfaz I2C que optimiza la integración con el microcontrolador.

### 3.5.2. Calibración de sensores

Cada sensor fue sometido a un proceso de calibración inicial para asegurar lecturas precisas:

- Para calibrar el sensor de pH, se utilizaron soluciones buffer de pH 4 y 6,86  $\pm 0,02$ , agua destilada y agua de red a temperatura ambiente. Primero, se limpió el sensor con agua destilada y se sumergió en un recipiente con agua de red. Luego se ajustó el potenciómetro a un valor de salida de 2,5 V (mitad de escala). A continuación, el sensor se sumergió en la solución de pH 4 durante un minuto y se ajustó el valor crudo al punto de correspondencia de pH. Este proceso se repitió con la solución de pH 6,86. Por último, se midió el agua de red para verificar que el valor reportado fuera cercano a 7,5.
- Para calibrar el sensor de conductividad, se utilizó agua destilada para marcar el valor de 0 mS/cm y una sonda patrón de conductividad para marcar el valor de 1 mS/cm mediante una solución de NaCl de 1 mS/cm a temperatura ambiente.
- Para calibrar el sensor de temperatura, se utilizó un termómetro patrón como referencia y se realizaron mediciones en dos puntos fijos: 0 °C, obtenido mediante una mezcla en equilibrio de hielo triturado y agua, y a aproximadamente 50 °C. Ambos instrumentos se colocaron a la misma profundidad, sin contacto con las paredes del recipiente y tras alcanzar el equilibrio térmico, para lograr así una calibración práctica y suficientemente precisa para las condiciones disponibles. Para mantener el sistema a una temperatura constante, se utilizaron 1,5 litros de agua en un termo de vidrio.
- Para calibrar el sensor de humedad, primero se realizó una medición con el sensor en sustrato seco y se estableció el valor en 0 %. Luego se sumergió completamente en agua y se ajustó el valor a 100 %.
- El sensor de luminosidad se utilizó con su calibración de fábrica, que proporciona mediciones precisas dentro de los rangos especificados por el fabricante.

## 3.6. Interfaz gráfica

Esta sección describe el diseño, estructura y funcionamiento de la interfaz gráfica desarrollada en React. El objetivo de la interfaz es proporcionar al usuario una visualización clara y en tiempo real de los parámetros del cultivo, además de ofrecer herramientas para el control y la configuración.

### 3.6.1. Diseño de la interfaz de usuario

El diseño se implementó siguiendo la metodología de diseño *mobile-first*, con un esquema de colores contrastantes que mejora la legibilidad de los valores mostrados. La interfaz se organiza en un diseño modular basado en tarjetas, donde cada sección agrupa información relacionada. Esta estructura no solo optimiza la navegación, sino que también facilita la identificación de los estados del sistema.

El núcleo App contiene dos componentes principales: Navbar y Router. Este último gestiona todas las vistas: Login para autenticación, Dashboard con gráficos y estados de sensores, Actuators para control de actuadores, Config para ajustes avanzados, y Logs con el historial de alertas. Además, se incluye el módulo AuthProvider, encargado de gestionar y proporcionar a los distintos componentes las credenciales de acceso al servidor embebido.

La figura 3.9 muestra la estructura jerárquica de la aplicación.

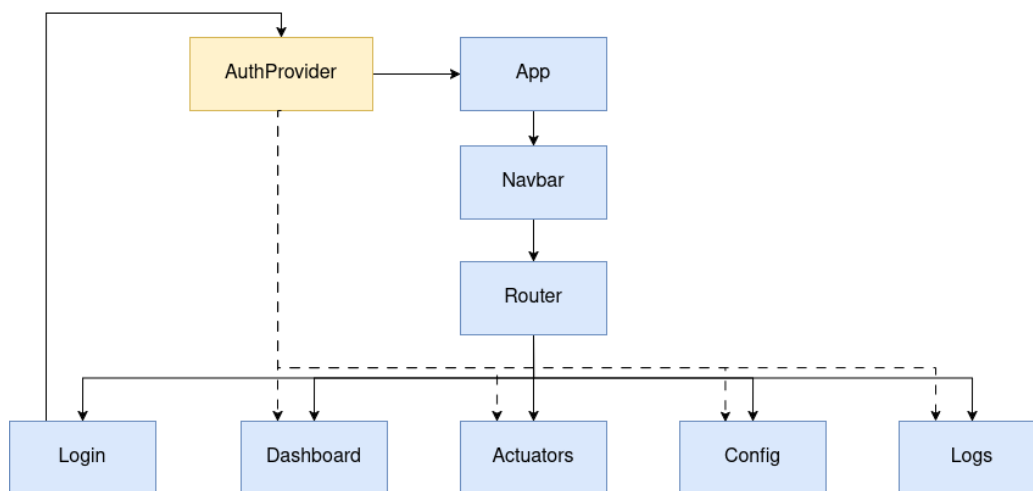


FIGURA 3.9. Diagrama de la estructura jerárquica de la aplicación.

Cuando el usuario navega entre las diferentes secciones, la aplicación actualiza automáticamente la ruta en la barra de navegación y carga el componente correspondiente. Este, a su vez, obtiene los datos iniciales necesarios de la API y renderiza sus subcomponentes. Para aquellas vistas que requieren sincronización periódica, se activa un bucle de actualización.

La figura 3.10 muestra la secuencia de renderizado de una vista.

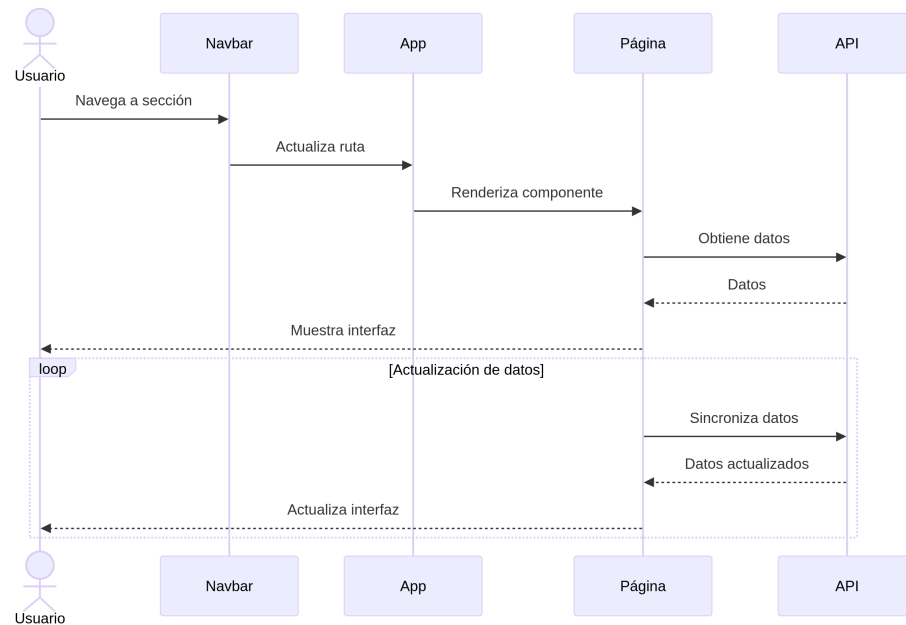


FIGURA 3.10. Diagrama de secuencia de renderizado de una vista.

### 3.6.2. Seguridad y autenticación

La seguridad de las comunicaciones se garantiza mediante un sistema de autenticación basado en tokens. Como se ilustra en la figura 3.11, cuando el usuario ingresa las credenciales, el componente `Login` envía una petición `POST` a la API. Si las credenciales son válidas, la API responde con una clave de acceso que el `AuthProvider` almacena en el `localStorage` del navegador. Finalmente, el usuario es redirigido al `Dashboard`.

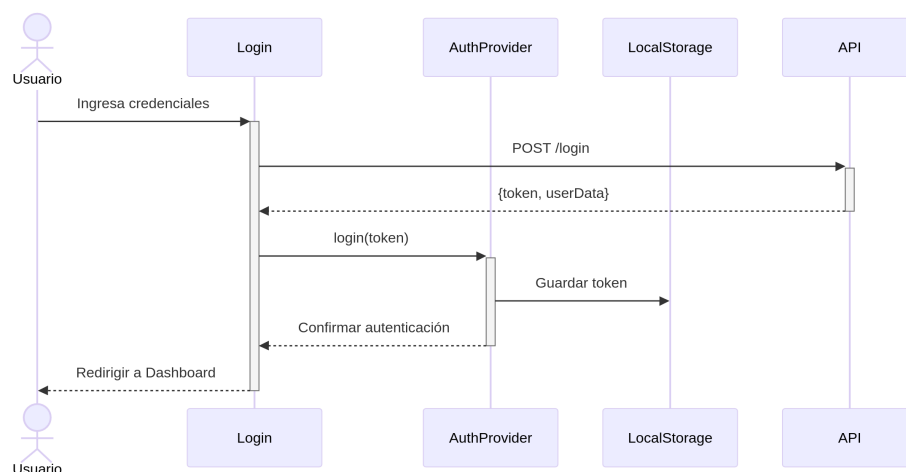


FIGURA 3.11. Diagrama de secuencia del proceso de autenticación.

### 3.6.3. Monitoreo de sensores

El panel principal muestra los valores medidos por los sensores. El componente `Sensors` realiza peticiones periódicas al backend embebido para obtener los datos más recientes cuando el usuario accede a esta sección. Cada sensor se presenta en una tarjeta que muestra su valor actual, unidad de medida y estado, con actualizaciones automáticas cada 1,5 segundos que garantizan la vigencia de la información. La interfaz también incluye indicadores visuales que muestran los estados de carga y los posibles errores en la comunicación con la API.

La figura 3.12 muestra el diagrama de secuencia que describe la interacción entre los componentes durante la carga y actualización de la vista de sensores.

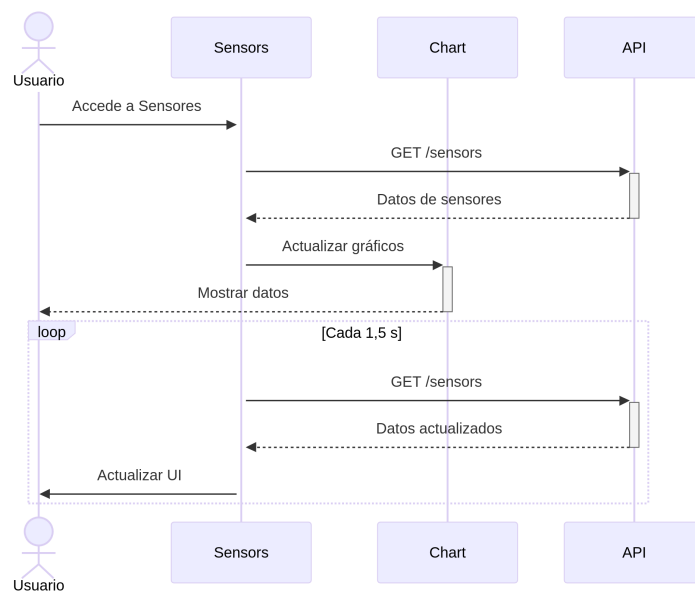


FIGURA 3.12. Diagrama de secuencia de la vista de monitoreo de sensores.

### 3.6.4. Control de actuadores

A través de esta vista, el usuario puede activar o desactivar manualmente los actuadores, como la bomba de riego, la iluminación y la ventilación, mientras visualiza el estado actual de cada uno de ellos (ver figura 3.13). Las acciones se transmiten al backend embebido mediante solicitudes `POST` y `GET` a los endpoints correspondientes.

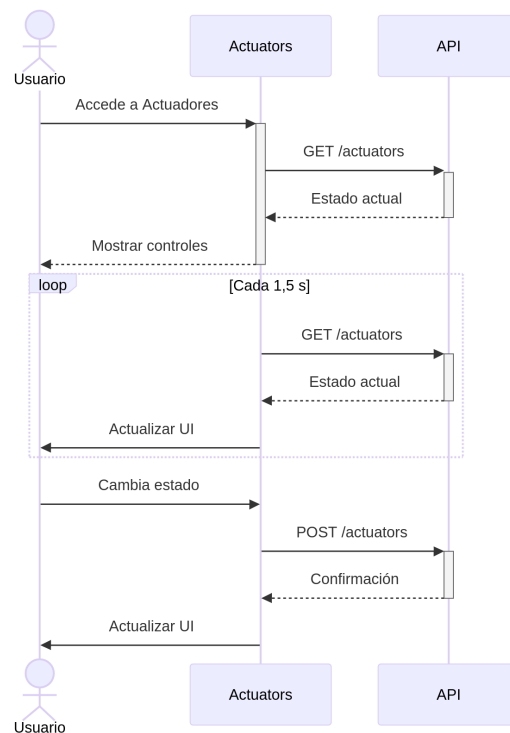


FIGURA 3.13. Diagrama de secuencia de la vista de control de actuadores.

### 3.6.5. Configuración del sistema

El panel de configuración permite modificar umbrales de alarma, períodos de riego o ventilación y horario de iluminación (ver figura 3.14). Una vez confirmados los cambios, el sistema actualiza el archivo `config.json` en memoria flash y propaga los nuevos valores a los controladores correspondientes.

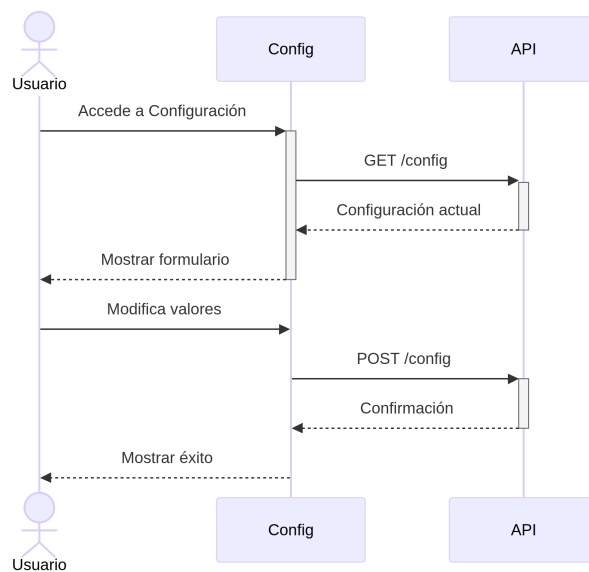


FIGURA 3.14. Diagrama de secuencia de la vista de configuración.

### **3.6.6. Logs del sistema**

Por último, el panel de logs muestra los eventos relevantes del sistema, como errores y advertencias. Además, incluye un botón de limpieza que permite vaciar el registro cuando el usuario lo requiera.

### **3.6.7. Compatibilidad y optimización**

La interfaz es completamente responsiva y se adapta a diferentes tamaños de pantalla. Se optimizó el rendimiento mediante la reducción de recursos estáticos, compresión `gzip` y almacenamiento en caché de datos locales. Estas optimizaciones minimizan el tiempo de carga y mejoran la experiencia del usuario incluso en conexiones inalámbricas inestables.



## Capítulo 4

# Ensayos y resultados

En este capítulo se presentan las pruebas y ensayos que se realizaron sobre el sistema de monitoreo y gestión de cultivos hidropónicos verticales desarrollado. El objetivo principal consistió en verificar el correcto funcionamiento del hardware, firmware y la interfaz de usuario, además de validar el cumplimiento de los requerimientos funcionales definidos en las etapas de diseño. Cabe destacar que acá se presentan casos de prueba representativos, seleccionados por su relevancia para demostrar el funcionamiento del sistema. La metodología descrita se aplicó de manera consistente en todos los ensayos realizados.

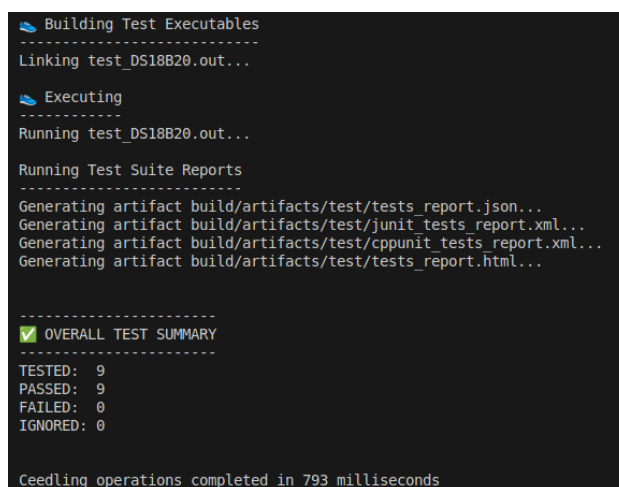
### 4.1. Pruebas del firmware

Antes de la implementación en la PCB, se validaron los componentes del firmware. Cada funcionalidad se evaluó de manera aislada mediante pruebas unitarias para verificar su funcionamiento antes de la integración del sistema.

#### 4.1.1. Pruebas unitarias

Se implementaron pruebas unitarias utilizando Ceedling sobre los *drivers* y componentes del firmware. Este enfoque permitió automatizar la ejecución de pruebas a medida que se fue desarrollando el código.

En la figura 4.1 se puede observar el resultado de las pruebas realizadas sobre el *driver* del sensor DS18B20.

A screenshot of a terminal window showing the output of Ceedling test runs. The text is as follows:

```
Building Test Executables
-----
Linking test_DS18B20.out...

Executing
-----
Running test_DS18B20.out...

Running Test Suite Reports
-----
Generating artifact build/artifacts/test/tests_report.json...
Generating artifact build/artifacts/test/junit_tests_report.xml...
Generating artifact build/artifacts/test/cppunit_tests_report.xml...
Generating artifact build/artifacts/test/tests_report.html...

-----
[✓] OVERALL TEST SUMMARY
-----
TESTED: 9
PASSED: 9
FAILED: 0
IGNORED: 0

Ceedling operations completed in 793 milliseconds
```

FIGURA 4.1. Reporte de pruebas unitarias de Ceedling.

### 4.1.2. Pruebas de comunicación entre módulos

#### Arquitectura pub-sub

La validación del sistema de mensajería confirmó la correcta distribución de mensajes a través de un buffer estático, donde cada suscriptor se identifica mediante su ID de hilo (ver figura 4.2). Las pruebas demostraron una gestión adecuada de semáforos durante la publicación y suscripción, lo que evita condiciones de carrera. Se comprobó la entrega confiable a múltiples destinatarios con preservación de la integridad de la información.

```
I (25375) pub_sub_subscribe: Subscribed to channel 'tds' subscriber: 0x3ffcc7a0
I (25385) pub_sub_subscribe: Subscribed to channel 'moist' subscriber: 0x3ffcc7a0
I (25395) pub_sub_subscribe: Subscribed to channel 'light' subscriber: 0x3ffcc7a0
I (25405) pub_sub_subscribe: Subscribed to channel 'u_config' subscriber: 0x3ffcc7a0
I (25415) pub_sub_subscribe: Subscribed to channel 'pump_act' subscriber: 0x3ffcc7a0
I (25425) pub_sub_subscribe: Subscribed to channel 'moist' subscriber: 0x3ffe3ca8
I (25435) pub_sub_subscribe: Subscribed to channel 'config' subscriber: 0x3ffe9828
I (25435) pub_sub_subscribe: Subscribed to channel 'config' subscriber: 0x3ffe86cc
I (25435) gpio: GPIO[14] InputEn: 0 OutputEn: 1 OpenDrain: 0 Pullup: 0 Pulldown: 0 Intr:0
I (25445) gpio: GPIO[12] InputEn: 0 OutputEn: 1 OpenDrain: 0 Pullup: 0 Pulldown: 0 Intr:0
I (25445) pub_sub_subscribe: Subscribed to channel 'fan_act' subscriber: 0x3ffcc7a0
I (25455) fan_controller: GPIO 14 configurado como salida
I (25465) light_controller: GPIO 12 configurado como salida
I (25475) pub_sub_subscribe: Subscribed to channel 'light_act' subscriber: 0x3ffcc7a0
I (25495) administrator_callback: Administrator is running
```

FIGURA 4.2. Proceso de suscripción a canales.

Como se muestra en la figura 4.3, cuando el buffer alcanza su capacidad máxima, el sistema descarta automáticamente los mensajes más antiguos sin interrumpir el flujo de comunicación. Este comportamiento se observa claramente al triplicar el tiempo entre lecturas, lo que permite verificar el correcto manejo de desbordamiento. Finalmente, se estableció una relación 1:1 entre publicadores y suscriptores, lo que permitió mantener estable cada canal y garantizar un flujo de información equilibrado.

```
D [*****-DEBUG-*****] pub_sub_publish: Message queue full for channel 'tds', oldest message
discarded, subscriber: 0x3ffcc7a0
I (51126) PH4502: ADC Raw: 181, Voltaje (mV*100): 14586, pH*100: 722
D [*****-DEBUG-*****] pub_sub_publish: Message queue full for channel 'ph', oldest message d
iscarded, subscriber: 0x3ffcc7a0
I (51156) moisture_monitor_task: Moisture: 100
D [*****-DEBUG-*****] pub_sub_publish: Message queue full for channel 'moist', oldest messag
e discarded, subscriber: 0x3ffcc7a0
D [*****-DEBUG-*****] fan_controller: interval_counter: 24
D [*****-DEBUG-*****] pub_sub_publish: Message queue full for channel 'tds', oldest message
discarded, subscriber: 0x3ffcc7a0
I (52126) PH4502: ADC Raw: 126, Voltaje (mV*100): 10153, pH*100: 723
D [*****-DEBUG-*****] pub_sub_publish: Message queue full for channel 'ph', oldest message d
iscarded, subscriber: 0x3ffcc7a0
I (52156) moisture_monitor_task: Moisture: 100
D [*****-DEBUG-*****] pub_sub_publish: Message queue full for channel 'moist', oldest messag
e discarded, subscriber: 0x3ffcc7a0
D [*****-DEBUG-*****] fan_controller: interval_counter: 25
```

FIGURA 4.3. Manejo de desbordamiento del buffer.

#### API REST con Postman

Se desarrollaron pruebas de integración para los endpoints REST expuestos por el sistema, organizados por funcionalidad.

A continuación, se listan los endpoints disponibles:

- Autenticación

- POST /v1/login: autenticación de usuario.
- Estado del sistema
  - GET /v1/status: estado general del sistema.
- Sensores
  - GET /v1/sensors: lectura de datos de sensores (temperatura, humedad, pH, CE, etc.).
- Actuadores
  - GET /v1/actuators: estado actual de los actuadores.
  - POST /v1/actuators: control de actuadores (bomba, ventilador, luces, calefactor).
- Configuración
  - GET /v1/config: obtener configuración actual.
  - POST /v1/config: actualizar configuración.
- Registros del sistema
  - GET /v1/logs: obtener registros del sistema.
  - DELETE /v1/log/clean: limpiar registros.

Todas las peticiones requieren autenticación mediante token, excepto el endpoint de login. El token debe incluirse en el encabezado `Authorization` de las peticiones posteriores a la autenticación.

En la figura 4.4 se muestra la respuesta del servidor al intentar acceder al endpoint de configuración sin autenticación, el sistema rechaza correctamente las peticiones que no incluyen un token de autenticación válido y devuelve un código de estado HTTP 401 (no autorizado) junto con un mensaje de error.

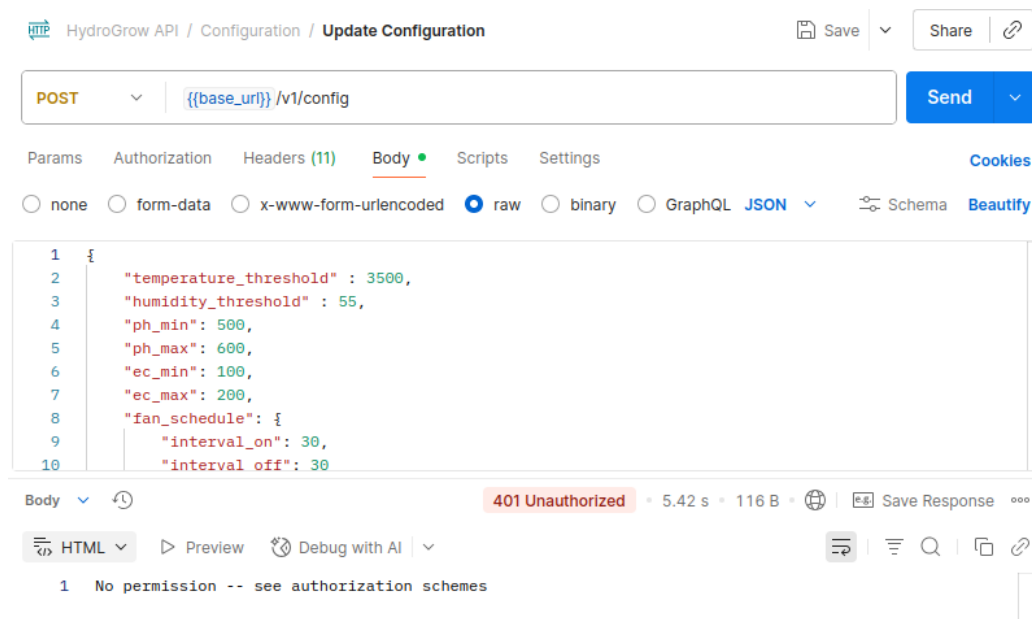


FIGURA 4.4. Respuesta del servidor al intentar acceder sin autenticación.

## Gestión de logs

Se verificó el sistema de rotación de logs, que incluyó la rotación automática al alcanzar el tamaño máximo configurado, la conservación de los registros más recientes según la política de retención establecida, el mantenimiento de un formato consistente en todos los mensajes de registro y la implementación de niveles de severidad apropiados (DEBUG, INFO, WARN, ERROR, ALERT) para una mejor clasificación y filtrado de los eventos registrados.

## Comunicación con la nube

Se implementó un sistema de envío de datos a un servidor backend mediante peticiones HTTP POST. El sistema utiliza un único endpoint para el envío de datos, con un esquema JSON que incluye todos los parámetros del sistema.

- Endpoint: `http://IP-raspberry-pi/api/v1/telemetry`.
- Método: POST.
- Autenticación: sin autenticación (en fase de pruebas).
- Formato de los datos: JSON.

En el código 4.1 se muestra el esquema JSON que se envía al servidor.

```
1 {  
2   "device_id": "hydrogrow-001",  
3   "timestamp": "2025-03-30T14:12:00Z",  
4   "sensors": {  
5     "temperature": 25.5,  
6     "humidity": 65.2,  
7     "ph": 6.2,  
8     "ec": 1.1,  
9     "light_intensity": 750,  
10    "water_level": "NORMAL"  
11  },  
12  "actuators": {  
13    "pump": false,  
14    "fan": true,  
15    "light": false,  
16    "heater": false  
17  },  
18  "system": {  
19    "uptime_seconds": 45234  
20  },  
21  "alerts": []  
22 }
```

CÓDIGO 4.1. Esquema JSON que se envía al servidor.

Con el fin de testear el dispositivo, se implementó un servidor de telemetría que recibe datos mediante una API REST. Los datos se guardan en una base de datos PostgreSQL. En esta versión inicial de pruebas no se implementó autenticación ni encriptación, pero están planificadas para versiones futuras.

Las pruebas en el módulo de control mostraron un funcionamiento correcto. El sistema envió datos cada 5 minutos con marcas de tiempo en formato ISO 8601. La sincronización entre módulos se mantuvo consistente, lo que permitió un monitoreo preciso de las variables ambientales y el estado de los actuadores.

### Estado del sistema y actuadores

Se implementaron pruebas para verificar la sincronización del estado entre tareas, se validaron las transiciones de estado, se aseguró la persistencia del estado ante reinicios inesperados y se comprobó el control de los actuadores según los comandos recibidos desde la interfaz de usuario.

### Pruebas de condiciones de carrera

El sistema se sometió a pruebas de condiciones de carrera que incluyeron la validación del acceso concurrente a recursos compartidos (variables globales, archivos y funciones de acceso a memoria), la verificación del manejo de prioridades de tareas, la confirmación del funcionamiento adecuado de las secciones críticas y mecanismos de sincronización, la evaluación de posibles situaciones de interbloqueo, así como la comprobación del manejo correcto de colas de mensajes en estados de lleno y vacío.

### Manejo de configuración

Se probó la generación, lectura y actualización del archivo `config.json`, lo que incluyó la creación con valores por defecto, la carga correcta de la configuración durante el inicio, la capacidad de actualizar parámetros de forma dinámica, la validación de los valores configurados, el manejo adecuado de archivos corruptos o faltantes y la persistencia ante reinicios inesperados. La figura 4.5 muestra la petición de actualización de configuración realizada con Postman, mientras que la figura 4.6 muestra la salida correspondiente en la consola de la ESP32 durante este proceso.

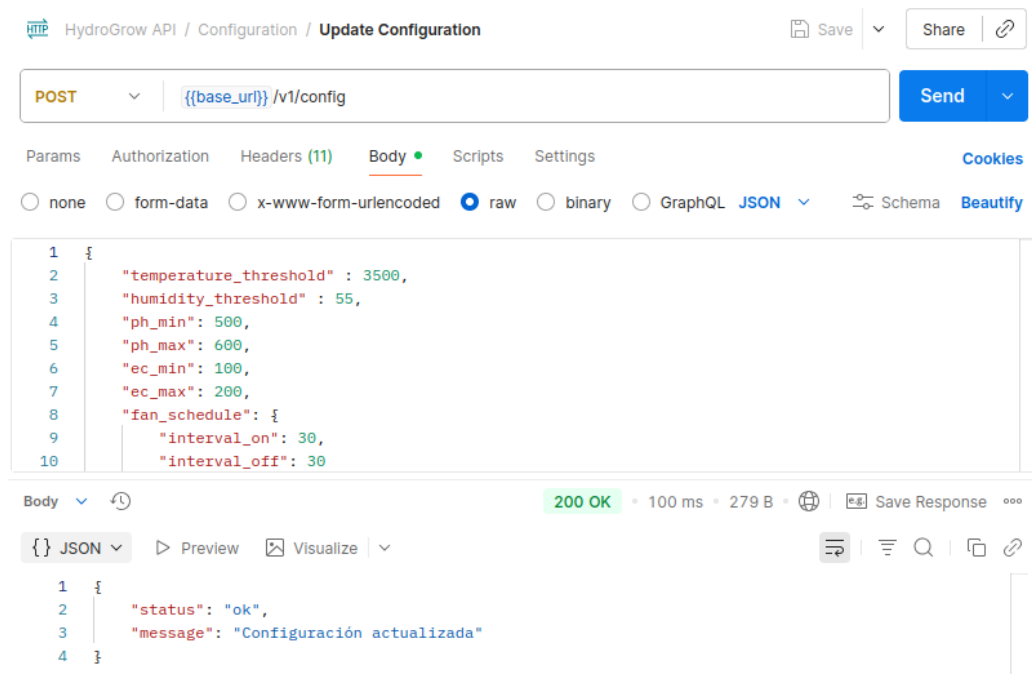


FIGURA 4.5. Pruebas de API con Postman - actualización de la configuración.

```

D [*****-DEBUG-*****] adminitrator: Se detecto un cambio en la configuracion
D [*****-DEBUG-*****] adminitrator: JSON serializado: 341 caracteres, copiado: 341 caractere
s
I (48466) adminitrator: Archivo de configuración actualizado exitosamente: config.json
D [*****-DEBUG-*****] adminitrator: Contenido actualizado del archivo "config.json": {"moist
ure_controller":{"humidity_th":55,"w_duration":25,"w_interval":2,"w_interval_mode":true,"w_start
_time":1225,"w_end_time":2210},"thresholds":{"temperature_th":3500,"ph_min":500,"ph_max":600,"ec
_min":100,"ec_max":200},"fan_controller":{"interval_on":30,"interval_off":30},"light_controller"
:{"light_time_on":1814,"light_time_off":2350}}
I (48496) temp_callback: Temperature: -6
D [*****-DEBUG-*****] adminitrator: Se recibió un update de configuración MOISTURE_UPDATE
D [*****-DEBUG-*****] adminitrator: Se recibió un update de configuración TEMP_UPDATE
D [*****-DEBUG-*****] adminitrator: Se recibió una opcion de configuracion FAN_UPDATE
D [*****-DEBUG-*****] adminitrator: Se recibió un update de configuración LIGHT_UPDATE

```

FIGURA 4.6. Salida por consola durante la actualización de la configuración.

## Conectividad de red

Las pruebas de conectividad verificaron la configuración inicial mediante WPS con el botón físico del dispositivo, que incluyó la asignación de dirección IP estática (192.168.0.100/24, con red estándar de gateway 192.168.0.x) en la red local y el almacenamiento persistente de credenciales en la memoria flash. En el fragmento de código 4.2, se muestra el resultado de la inicialización del sistema y se aprecia la configuración de la IP estática junto con la configuración WPS.

- 1 I (856) wifi: Iniciando WPS...
- 2 I (866) wifi: WIFI\_EVENT\_STA\_START (WPS mode)
- 3 I (28736) wifi: WPS exitoso
- 4 I (28746) wifi: WiFi credentials saved to filesystem
- 5 I (32436) wifi: Gateway aprendido: 192.168.0.1. Reconfigurando con IP estática...
- 6 I (32436) adminitrator: Conectado exitosamente
- 7 I (34446) wifi: Reconectando con IP estática usando gateway: 192.168.0.1
- 8 I (35456) wifi: Configurando IP estática: IP=192.168.0.100, GW=192.168.0.1, ↘  
→NETMASK=255.255.255.0
- 9 I (42446) web\_server: HTTP Server started
- 10 I (42446) hydro\_rest: Todos los endpoints REST registrados correctamente

CÓDIGO 4.2. Resultado de la inicialización del sistema tomado de la consola de depuración.

## 4.2. Pruebas del hardware

Se realizaron pruebas funcionales sobre la PCB desarrollada y los componentes asociados, con el objetivo de validar la operación eléctrica correcta y la integridad de las señales entre módulos.

### 4.2.1. Verificación eléctrica

Se midieron las tensiones de alimentación y los niveles lógicos en cada etapa. Los resultados se mantuvieron dentro de los márgenes de diseño (5,1 V y 3,3 V regulados). Las salidas de actuadores se probaron mediante una lámpara para evaluar de forma visual la acción de cada controlador. Además, se verificó el funcionamiento correcto de los LEDs indicadores integrados en la placa para cada actuador.

#### 4.2.2. Validación funcional

Se realizaron pruebas eléctricas de continuidad y verificación de polaridades antes de alimentar el circuito. La placa se energizó sin los módulos y se midieron las tensiones en todos los puntos de alimentación para validar el funcionamiento correcto de los reguladores. Posteriormente, se realizaron pruebas funcionales para validar la operación correcta de los sensores, actuadores y módulos de comunicación. Los resultados confirmaron el correcto funcionamiento del sistema en condiciones normales de operación.

#### 4.2.3. Pruebas de protección

Se verificó el correcto funcionamiento del sistema de protección contra sobrecorriente. Tras desconectar los módulos de la PCB, se conectó una carga variable en serie con un amperímetro. Este montaje permitió incrementar gradualmente la corriente mientras se monitoreaba la tensión de salida. El umbral de corte se activó en  $0,5 \pm 0,1$  A, valor acorde a las especificaciones del componente. La corriente de fuga en condiciones normales se mantuvo dentro de los márgenes de seguridad, y el fusible auto-resetable restableció la operación al normalizarse la corriente.

### 4.3. Pruebas de la interfaz gráfica

Se realizaron pruebas de la interfaz gráfica que abarcaron todas sus funcionalidades. Dichas pruebas incluyeron la validación de la interfaz de usuario, la responsividad en diferentes dispositivos y resoluciones, la navegación entre secciones, el correcto envío y recepción de datos al servidor embebido y el manejo adecuado de casos de error.

#### 4.3.1. Pruebas de autenticación

Se validó el proceso de inicio de sesión mediante credenciales. La figura 4.7 muestra la pantalla de autenticación, que incluye campos para usuario y contraseña. La interfaz mostró un comportamiento consistente en diferentes tamaños de pantalla y se adaptó correctamente a dispositivos móviles y de escritorio.

#### 4.3.2. Monitoreo y actualización de datos

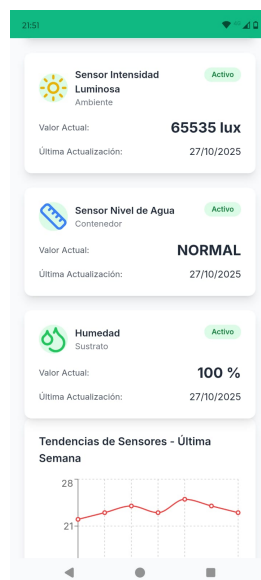
Cada magnitud se mostró correctamente en una tarjeta independiente con su valor, unidad, icono identificativo y fecha de actualización. La figura 4.8 muestra esta disposición organizada.



FIGURA 4.7. Pantalla de inicio de sesión del sistema.



(A) Vista de sensores 1.



(B) Vista de sensores 2.



(C) Gráficos de sensores.

FIGURA 4.8. Vistas de monitoreo de sensores.

### 4.3.3. Control de actuadores

Se validó la vista de control de actuadores con tres componentes: bomba de agua, ventilador y luces. Cada uno mostró su estado en la parte superior y un botón que cambia de color (rojo para apagar, verde para encender). La figura 4.9 muestra la bomba apagada, y el ventilador y luces encendidos.

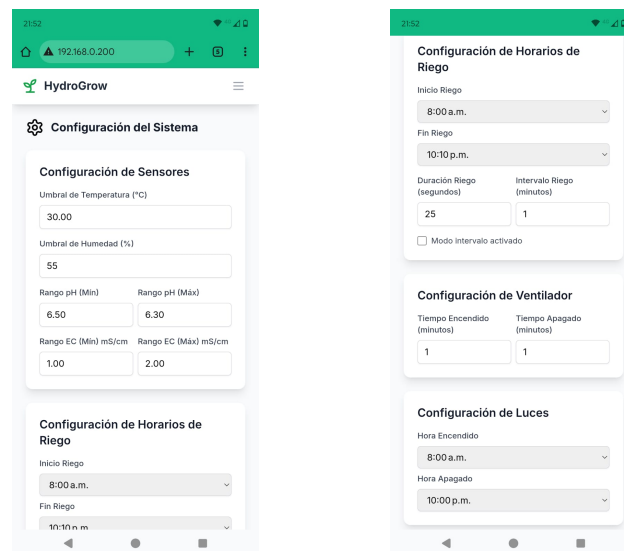




FIGURA 4.9. Interfaz de control de actuadores.

#### 4.3.4. Configuración del sistema

La interfaz permitió modificar la configuración del sistema a través de un formulario con múltiples campos de entrada. Cada campo incluyó validación básica y mostró el teclado correspondiente según el tipo de dato requerido. La figura 4.10 muestra las vistas de configuración disponibles.



(A) Menú de configuración superior.

(B) Menú de configuración inferior.

FIGURA 4.10. Pantallas de configuración.

#### 4.3.5. Registro de eventos

La interfaz incluye un registro de eventos del sistema, como se muestra en la figura 4.11.

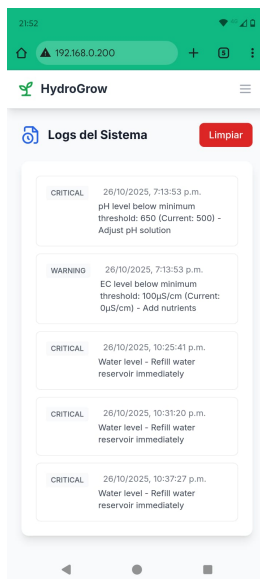


FIGURA 4.11. Registro de eventos del sistema.

#### 4.4. Pruebas de integración

Una vez validados los módulos individuales, se realizaron pruebas de integración para verificar el funcionamiento conjunto del hardware, firmware e interfaz web, con el objetivo de analizar la comunicación y sincronización entre las tareas del firmware (monitores, controladores y servidor embebido). Estas pruebas permitieron validar la interacción entre los componentes del sistema, la estabilidad de la comunicación en condiciones controladas, la coherencia de los datos transmitidos y la correcta ejecución de las acciones automáticas. En la figura 4.12 se muestra la vista de sensores de la interfaz web, donde se pueden visualizar los valores medidos en tiempo real.

Las pruebas se realizaron siguiendo un procedimiento estructurado que comenzó con la conexión de sensores y actuadores al módulo principal, seguido de la configuración de la interfaz web para monitoreo y control remoto. Se aplicaron variaciones controladas en las variables ambientales (temperatura, humedad y pH) para evaluar el rendimiento del sistema. Los resultados mostraron una respuesta adecuada a las variaciones simuladas, con una comunicación estable entre módulos y registro preciso de datos en la interfaz. Durante las pruebas, se verificó la correspondencia exacta entre los valores medidos y las acciones ejecutadas, sin pérdida de información en ningún momento.

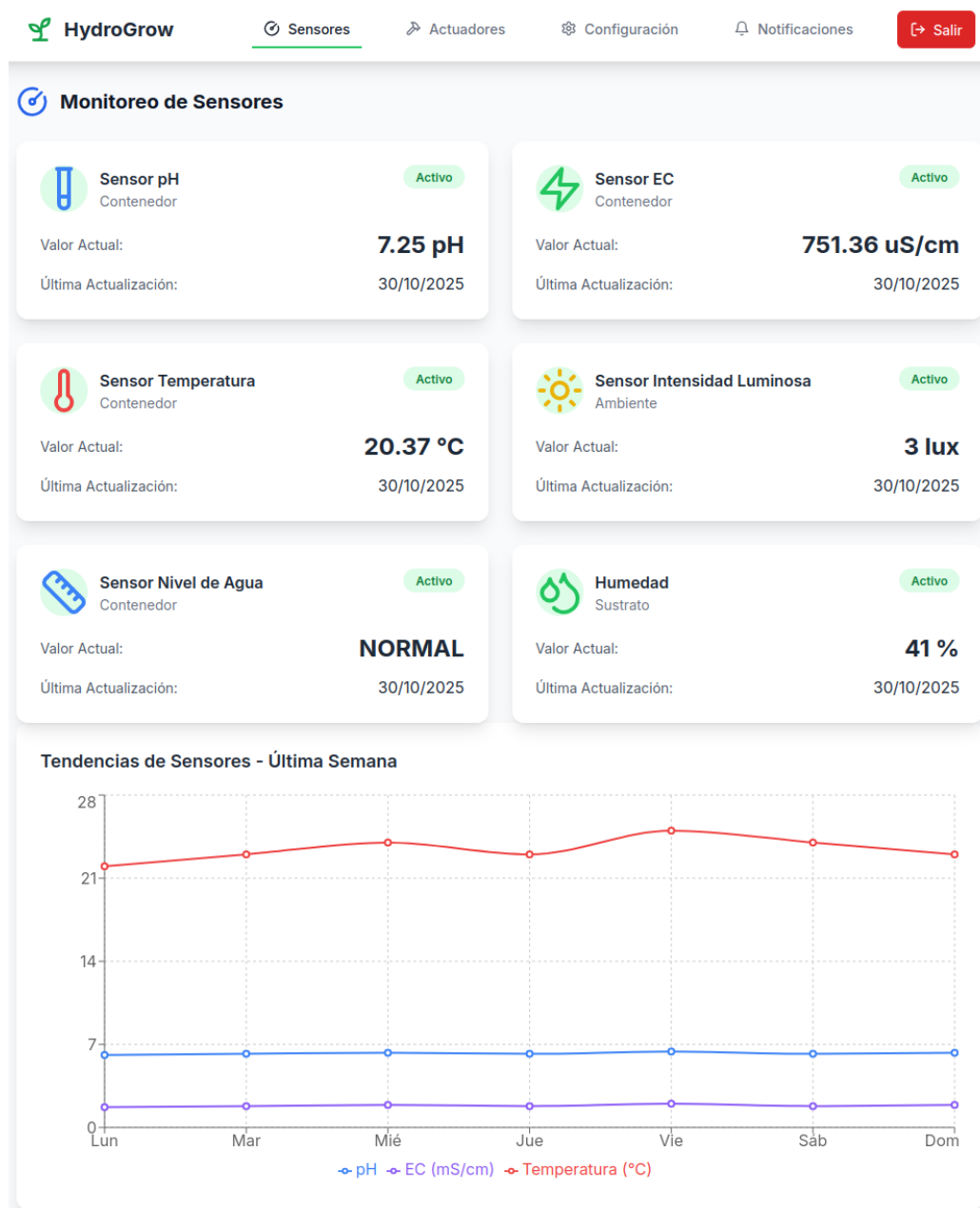


FIGURA 4.12. Vista de sensores con mediciones en tiempo real.

## 4.5. Validación de requerimientos funcionales

Esta etapa tuvo por objetivo asegurar que el sistema cumple con los requerimientos definidos durante la planificación del trabajo.

A continuación, se detallan los requerimientos funcionales validados y las pruebas realizadas para cada uno:

- Control automático de riego, iluminación y oxigenación: se verificó que el sistema pueda controlar el estado de los actuadores según las condiciones ambientales monitoreadas y los parámetros de control definidos. Las pruebas incluyeron cambios en las condiciones ambientales, donde se midió el tiempo de respuesta del sistema desde la detección de la variación hasta la

activación del actuador. Utilizando un cronómetro, se registró un tiempo de respuesta promedio de  $2,0 \pm 0,3$  segundos en 25 mediciones consecutivas, lo que cumple con el requisito de respuesta en menos de 5 segundos. Los resultados demostraron una activación consistente y confiable de los sistemas de riego, iluminación y oxigenación en respuesta a las condiciones ambientales monitoreadas y los parámetros de control.

- **Monitoreo de variables ambientales:** se validó la precisión y confiabilidad del sistema de adquisición de datos mediante la comparación con instrumentos de referencia calibrados. Se utilizó un termómetro digital (precisión  $\pm 0,1$  °C), dos soluciones buffer de pH (precisión  $\pm 0,02$ ) y un conductímetro de mercado (precisión  $\pm 2 \mu\text{S}/\text{cm}$ ) como estándares de referencia. Las mediciones se realizaron en condiciones controladas de laboratorio, con un banco de pruebas que simulaba las condiciones del cultivo. Se evaluaron diferentes configuraciones de parámetros ambientales, que incluyeron temperatura, humedad y composición de la solución nutritiva. Las mediciones mostraron una concordancia aceptable con los instrumentos de referencia, lo que confirmó la precisión del sistema de medición en condiciones controladas. Todos los valores medidos se mantuvieron dentro de los rangos especificados: temperatura ( $\pm 0,8$  °C), humedad del sustrato (sin patrón de referencia,  $\pm 2,5$  % de precisión según especificaciones del fabricante), nivel de solución nutritiva (mínimo 3/4 del contenedor), conductividad eléctrica ( $\pm 0,1$  mS/cm) y pH ( $\pm 0,1$ ).
- **Activación manual de secuencias de riego:** se realizaron 10 pruebas consecutivas en las que se midió el tiempo de respuesta desde la interacción del usuario en la interfaz hasta la activación del sistema de riego. Los resultados mostraron un tiempo promedio de respuesta de 1,2 segundos, lo que cumple con el requerimiento de menos de 2 segundos.
- **Servidor embebido:** se realizaron pruebas de disponibilidad en diferentes momentos del día. El servidor se mantuvo en funcionamiento continuo durante 72 horas, período en el que se monitoreó su rendimiento con peticiones HTTP cada 2 segundos. El sistema demostró un 100 % de disponibilidad.
- **Interfaz gráfica accesible desde móviles:** las pruebas en diferentes dispositivos móviles (Android) y navegadores (Chrome, Firefox) mostraron tiempos de carga promedio de 200 ms en conexiones Wi-Fi, lo que cumple con el requerimiento de menos de 5 segundos.
- **Programación de secuencias de riego:** se configuraron múltiples secuencias de riego con diferentes parámetros (por tiempo, humedad del sustrato, horarios específicos). La precisión de ejecución se verificó con un reloj atómico como referencia, lo que permitió alcanzar una precisión de  $\pm 1$  segundo, con lo que se superó el requerimiento de  $\pm 1$  minuto.

## 4.6. Banco de pruebas y resultados obtenidos

El banco de pruebas, mostrado en la figura 4.13, se configuró en un entorno controlado con sensores reales y un circuito de carga equivalente utilizando una lámpara. Las pruebas incluyeron la simulación de condiciones ambientales como humedad, pH, electroconductividad y variaciones de temperatura, además de pruebas de conexión Wi-Fi intermitente y verificación de alarmas ante el alcance de los umbrales establecidos por configuración. El sistema demostró estabilidad operativa continua durante 2 horas, sin pérdida de datos ni bloqueos.

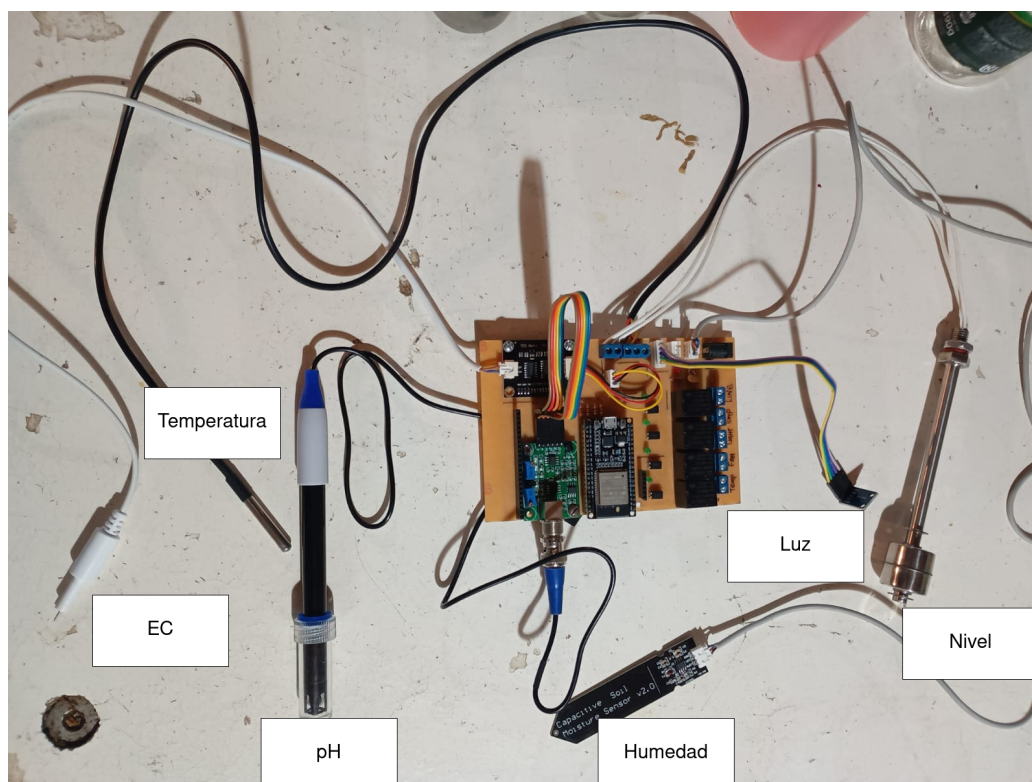


FIGURA 4.13. Configuración del banco de pruebas con los componentes principales del sistema.

### 4.6.1. Resumen general

Los resultados de las pruebas confirman que el sistema cumple con los requisitos de diseño tanto a nivel de hardware como de firmware, con un funcionamiento estable que se mantuvo ininterrumpido durante el período de pruebas sin registrar incidentes.



## Capítulo 5

# Conclusiones

En el presente capítulo se presentan las conclusiones sobre el trabajo realizado, las herramientas y conocimientos adquiridos durante el desarrollo, y se proponen mejoras para futuras versiones del sistema. Además, se destacan las competencias académicas aplicadas durante la implementación del proyecto.

### 5.1. Conclusiones generales

En el trabajo realizado se logró implementar de forma exitosa un prototipo de sistema de monitoreo y control para cultivos hidropónicos verticales. A través de las pruebas realizadas se pudo validar y demostrar su correcto funcionamiento. Se pueden nombrar los siguientes logros obtenidos:

- Se desarrolló e implementó un prototipo funcional que cumple con todos los requerimientos establecidos inicialmente, con capacidad para monitorear las variables ambientales de interés (temperatura, humedad, pH y conductividad eléctrica) y controlar los actuadores para el manejo del cultivo.
- Se diseñó y fabricó una PCB personalizada que integra todos los componentes electrónicos necesarios, con circuitos de acondicionamiento de señal para los sensores y etapas de potencia para los actuadores. El diseño se desarrolló con KiCad y se basó en buenas prácticas de diseño electrónico.
- Se implementó el firmware sobre FreeRTOS, lo que permitió optimizar los recursos del microcontrolador y desarrollar un código modular, escalable y mantenible. Se utilizaron colas y semáforos para la comunicación y sincronización entre tareas, lo que resultó en tiempos de respuesta promedio de  $2,0 \pm 0,3$  segundos en la activación de actuadores.
- Se desarrolló un servidor web embebido con API REST que permite el monitoreo y control remoto del sistema. Se implementó autenticación segura, se configuró el soporte para CORS (*Cross-Origin Resource Sharing*) para permitir solicitudes desde diferentes orígenes, y se validó el funcionamiento mediante pruebas con Postman.
- Se implementaron pruebas unitarias con Ceedling, bajo la metodología de desarrollo guiado por pruebas (TDD) que garantiza la calidad del código y facilita el mantenimiento futuro.

### 5.2. Próximos pasos

Las siguientes mejoras podrían implementarse en futuras versiones del sistema:

- Mejoras en el hardware:
  - Rediseñar la PCB para integrar todos los componentes en una sola placa, lo que elimina la necesidad de módulos externos.
  - Añadir protección contra sobretensiones y picos de corriente para aumentar la confiabilidad del sistema.
- Actualizaciones de firmware:
  - Implementar actualización de firmware por vía inalámbrica para facilitar el mantenimiento.
  - Añadir cifrado en las comunicaciones para mejorar la seguridad de los datos.
  - Optimizar el consumo de energía mediante la implementación de modos de bajo consumo.
- Nuevas funcionalidades:
  - Incorporar sensores adicionales como CO<sub>2</sub>, oxígeno disuelto y radiación fotosintética.
  - Desarrollar una aplicación móvil nativa para facilitar el monitoreo y control desde dispositivos móviles.
  - Implementar un sistema de alertas por SMS o notificaciones push para avisos urgentes.
  - Desarrollar un sistema de control automático de la solución nutritiva, con monitoreo de pH y conductividad eléctrica.
- Mejoras en la escalabilidad:
  - Desarrollar protocolos de comunicación de mayor alcance como LoRa o Sigfox para zonas rurales.
  - Crear una arquitectura distribuida que permita gestionar múltiples unidades de cultivo de forma centralizada.
  - Implementar compatibilidad con estándares de agricultura de precisión para facilitar la integración con otros sistemas.
- Optimización de recursos:
  - Implementar algoritmos de aprendizaje automático para optimizar el uso de agua y nutrientes.
  - Desarrollar un sistema de recomendación de nutrientes basado en el estado actual del cultivo.
  - Crear modelos predictivos para anticipar necesidades de riego y fertilización.

Estas mejoras permitirían transformar el prototipo actual en un sistema comercialmente viable, con características avanzadas de monitoreo y control que podrían ser de gran utilidad para agricultores profesionales y entusiastas de la hidroponía por igual.



# Bibliografía

- [1] UN. «Población». En: *www.un.org* (2023). URL: <https://www.un.org/es/global-issues/population>.
- [2] Miguel A. Gómez Carlos Miguel Marschoff Marisa Arienza Andrés E. Carlsen Pittaluga. «AGUA». En: *www.greencross.org.ar* (). URL: <https://www.greencross.org.ar/download/LibroAguaaimprirweb.pdf>.
- [3] FAO. «El estado de la seguridad alimentaria y la nutrición en el mundo 2022». En: *fao.org* (2022). URL: <https://openknowledge.fao.org/items/ec2794d0-7265-47d8-a9c1-439c7d836a50>.
- [4] Banco Santander. *Cultivos Verticales: Ventajas y Desventajas*. 2023. URL: <https://www.bancosantander.es/blog/pymes-negocios/cultivos-verticales-ventajas-desventajas#:~:text=Los%20cultivos%20verticales%20son%20una,del%20espacio%20y%20la%20luz..>
- [5] Debajyoti Saha Aditi Saha Roy Saptashree Das y Subhajit Barat. «Vertical farming: A sustainable agriculture format of the future». En: *International Journal of Research in Agronomy* (2024). URL: <https://doi.org/10.33545/2618060X.2024.v7.i4Sd.641>.
- [6] Agroquivir. *Agricultura vertical: Cómo funciona y tipos*. 2023. URL: <https://agroquivir.com/agricultura-vertical-como-funciona-y-tipos/>.
- [7] Clarín. *Se desperdicia el 84 por ciento del agua para riego*. 2024. URL: [https://www.clarin.com/sociedad/desperdicia-84-ciento-agua-riego\\_0\\_rJ-xuMJZCtg.html](https://www.clarin.com/sociedad/desperdicia-84-ciento-agua-riego_0_rJ-xuMJZCtg.html).
- [8] Agrotec. *Nidopro - Sistema Hidropónico Inteligente de Cultivo Vertical*. 2025. URL: <https://www.agrotec.com/producto/nidopro-sistema-hidroponico-inteligente-cultivo-vertical/>.
- [9] Techpunt. *Xiaomi Mi Flower Care Plant Sensor*. 2025. URL: <https://www.techpunt.nl/en/xiaomi-mi-flower-care-plant-sensor.html>.
- [10] Konyks. *Hydro Kit*. 2025. URL: <https://konyks.com/es/producto/hydro-kit/>.
- [11] Click and Grow. *The Smart Garden 9 Pro*. 2025. URL: <https://tinyurl.com/ywvwxya9>.
- [12] Espressif Systems. *ESP32 DevKitC User Guide*. 2025. URL: [https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user\\_guide.html#functional-description](https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html#functional-description).
- [13] Think Robotics. *PH4502C PH Meter*. 2025. URL: <https://tinyurl.com/2nsm49rr>.
- [14] Ichibot. *TDS Meter V1 Module - Water Meter Filter Measuring Water Quality Sensor*. 2025. URL: <https://store.ichibot.id/product/tds-meter-v1-module-water-meter-filter-measuring-water-quality-sensor/>.
- [15] ProBots. *Soil Moisture Sensor Capacitive V1.2*. 2025. URL: <https://probots.co.in/soil-moisture-sensor-capacitive-v1-2.html>.
- [16] Analog Devices. *DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet*. 2025. URL: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>.

- [17] Mouser Electronics. *BH1750FVI Digital Light Sensor Datasheet*. 2025. URL: <https://tinyurl.com/87pka5mw>.
- [18] ARM. *Architecture*. 2025. URL: <https://www.arm.com/architecture>.
- [19] Raspberry Pi Foundation. *Getting started*. 2025. URL: <https://tinyurl.com/34ker5vm>.
- [20] Wikipedia. *GPIO*. 2025. URL: <https://es.wikipedia.org/wiki/GPIO>.
- [21] Wikipedia. *I<sup>2</sup>C*. 2025. URL: <https://es.wikipedia.org/wiki/I%C2%B2C>.
- [22] Wikipedia. *Serial Peripheral Interface*. 2025. URL: [https://es.wikipedia.org/wiki/Serial\\_Peripheral\\_Interface](https://es.wikipedia.org/wiki/Serial_Peripheral_Interface).
- [23] Analog Devices. *UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter*. 2020. URL: <https://sl1nk.com/VJxpb>.
- [24] Wikipedia. *Ethernet*. 2011. URL: <https://es.wikipedia.org/wiki/Ethernet>.
- [25] Wikipedia. *Wifi*. 2025. URL: <https://es.wikipedia.org/wiki/Wifi>.
- [26] Espressif Systems. *ESP-IDF: Espressif IoT Development Framework*. 2025. URL: <https://www.espressif.com/en/products/sdks/esp-idf>.
- [27] FreeRTOS. *FreeRTOS: Real-Time Operating System*. 2025. URL: <https://www.freertos.org/>.
- [28] Throw The Switch. *Ceedling: A Build System and Test Framework for C*. 2025. URL: <https://www.throwtheswitch.org/ceedling>.
- [29] React. *React*. 2025. URL: <https://es.react.dev/>.
- [30] Docker. *Introduction*. 2025. URL: <https://docs.docker.com/get-started/introduction/>.
- [31] Mozilla Developer Network (MDN). *Protocolo de Transferencia de Hipertexto (HTTP)*. 2025. URL: <https://developer.mozilla.org/es/docs/Web/HTTP>.
- [32] Wikipedia. *1-Wire*. 2025. URL: <https://es.wikipedia.org/wiki/1-Wire>.